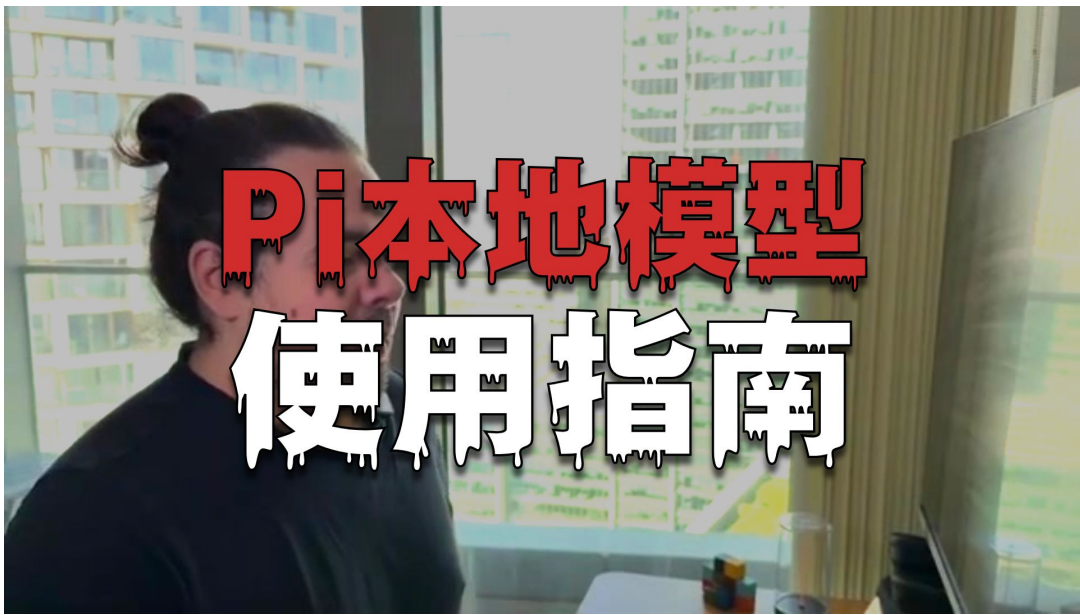


Bilibili 技术讲义

Pi Agent：程序员的极简主义宣言

视频作者：bushacker 整理：Codex

2026 年 5 月 12 日



视频标题：Pi Agent：程序员的极简主义宣言

视频链接：<https://www.bilibili.com/video/BV1o5RWBaE16/>

视频分辨率：640×360。讲义中的源码和终端小字会用重绘图、表格和文字解释补足。

字幕来源：B 站 ai-zh 自动字幕，已按章节时间边界重构为教学笔记。

目录

1	Pi Agent 的极简内核：少即是多	4
1.1	从 Claude Code 剥到核心	4
1.2	一句 system prompt 与四类工具	4
1.3	上下文窗口为何是稀缺资源	5
1.4	MCP、sub-agent 与 plan mode 的按需外接	5
1.5	sandbox、tmux 与 Markdown 计划文件	6
1.6	关键画面与结构图	7
1.7	本章小结	9
2	安装、Provider 与模型自由	9
2.1	安装与首次启动	10
2.2	Provider 解耦：订阅、OpenRouter 与 API key	10
2.3	模型切换、fork 与本地模型	11
2.4	常用快捷键与命令入口	11
2.5	配置文件夹的总体地图	12
2.6	关键画面与结构图	12
2.7	本章小结	14
3	Session 树：可恢复、可分叉、可审计的对话历史	15
3.1	Session 文件为何像工程日志	15
3.2	tree：把对话看成分支结构	15
3.3	fork：从历史节点另开实验路径	16
3.4	恢复、分享、导出与统计	16
3.5	会话树带来的协作价值	17
3.6	关键画面与结构图	18
3.7	本章小结	20
4	Context Engineering：把该进上下文的东西放进去	20

目录	2
4.1 上下文工程的基本问题	20
4.2 AGENTS.md: 项目级说明书	21
4.3 system.md 与 append-system.md: 角色和规则注入	21
4.4 Compaction 的收益与损耗	22
4.5 Skills 与 Prompt 模板的版本化	22
4.6 关键画面与结构图	23
4.7 本章小结	25
5 Extensions、Packages 与可编程安全边界	26
5.1 Extension 为什么是 Pi 的扩展中枢	26
5.2 事件、命令与工具: 从 Session Started 到 Weather Tool	26
5.3 把安全策略写成拦截逻辑	27
5.4 从 examples 到自举式扩展开发	27
5.5 Packages 与 Themes: 生态和外观的可定制性	28
5.6 关键画面与结构图	28
5.7 本章小结	33
6 多种集成形态与社区分支: 把 Pi 改造成自己的工具	33
6.1 从交互式 CLI 到批处理	34
6.2 JSON Event Stream 与 RPC	34
6.3 SDK: 从零构造自己的 Agent Session	34
6.4 社区分支与语言移植	35
6.5 最终综合: 极简主义怎样变成可塑性	36
6.6 关键画面与结构图	36
6.7 本章小结	42
7 总结与延伸: 极简主义怎样落到工程手上	42
7.1 从一个极简内核长出完整工作流	43
7.2 上下文预算是这套设计的隐含账本	43

7.3 安全、并行和计划都可以外接	44
7.4 实践上的取舍	44
7.5 本章小结	44

1 Pi Agent 的极简内核：少即是多

本章要解决的问题很直接：Pi Agent 为什么把一个编码 agent 缩到这么小，只保留极短的 system prompt 和少量文件、编辑、命令工具？这件事表面上像功能裁剪，实质上是在重新安排三类资源：模型已经学会的能力、上下文窗口里的 token 预算，以及 Unix 工具链本来就能承担的工程职责。

1.1 从 Claude Code 剥到核心

Pi Agent 是一个以极简工具集、短 system prompt 和可扩展能力为特征的编码 agent。这里的 agent 指能围绕用户目标持续读取材料、修改文件、调用工具并根据结果调整行动的软件助手；编码 agent 则把这种循环放进代码仓库、终端命令和开发文档之中。

视频一开始把 Pi Agent 放在 Claude Code 的对照面上。Claude Code 是功能更完整、脚手架更多的编码 agent；Pi Agent 的取向是把周边结构拆薄，只留下让模型完成编程任务所需的核心表面。这个边界在于：Claude Code 更像带有大量内建流程的工程座舱，Pi Agent 更像一把能进入 shell、文件系统和扩展接口的短刀。前者把许多 workflow 预装进产品，后者把 workflow 交还给普通文件、命令和用户可改造的配置。

P

i Agent 的“少”并不等于能力贫乏。更准确地说，它假设现代 LLM 已经在训练中学会了常见编码循环：搜索文件、读代码、编辑、运行测试、观察错误、再修正。工具层只要提供读、写、编辑和 bash 这几块地基，模型就能把地基组合成复杂动作。

这里的 LLM 是 Large Language Model，即大语言模型。它在一次推理中接收文本、工具说明、历史消息和规则，再生成下一步回答或工具调用。Pi Agent 的极简主义建立在一个判断上：模型行为的一部分已经被训练阶段内化，产品层无需反复用长规则描述同一套基础编码动作。

1.2 一句 system prompt 与四类工具

system prompt 是写给模型的顶层角色和行为指令，类似剧场开演前给演员的角色设定，也像操作系统里最先加载的全局配置。Pi Agent 默认 system prompt 极短，核心约束接近一句话：模型是一名专业编码助手。短 system prompt 把更多上下文空间留给当前任务、仓库文件和用户真正提供的规则。

Pi Agent 的基础工具可以概括为四类：读取文件、写入文件、编辑文件、执行 bash 命令。读取让模型获得证据，写入和编辑让模型改变仓库状态，bash 让模型运行测试、构

建、脚本和系统工具。四类工具合起来形成一个小闭环：

观察代码 → 修改文件 → 运行命令 → 读取反馈 → 继续修正。

强化学习（RL, Reinforcement Learning）是训练阶段通过反馈塑造模型行为的机制。可以把它想成“做题、评分、再调整策略”的循环：模型给出行动，系统或人类反馈告诉它哪些行动更好，训练过程提高好行动出现的概率。post-training 指预训练之后的对齐、强化学习和任务优化等过程。视频中强调，现代编码模型已经在这些阶段反复见过“改代码、跑命令、修 bug”的路径，所以 Pi Agent 不急着把每一步流程都写进厚重脚手架。

1.3 上下文窗口为何是稀缺资源

上下文窗口是模型一次推理能接收的 token 空间。token 是模型处理文本的计量单位，粗略可以理解为文字被切成的小片。上下文窗口像一张工作台：工具说明、历史对话、项目规则、报错日志、代码片段都会摆上去；工作台越拥挤，留给当前问题的空间越少。

上下文预算可以用一个轻量公式理解：

$$C_{usable} = C - C_{mcp} - C_{history} - C_{rules}$$

其中， C 表示总上下文窗口； C_{mcp} 表示 MCP server 描述占用的 token； $C_{history}$ 表示历史消息占用的 token； C_{rules} 表示规则文件、system prompt 和项目指令占用的 token； C_{usable} 才是当前任务真正可用的剩余空间。这个式子只是工程直觉模型，不用于精确预测性能。

上

下文窗口的关键取舍像背包问题：每放入一份工具说明或历史记录，就少放一份当前代码、错误日志或设计约束。Pi Agent 的极简设计是在问一个朴素问题：这份内容是否真的比当前任务证据更值得占据模型注意力？

这一点解释了 Pi Agent 对 MCP 的谨慎态度。MCP 是 Model Context Protocol，用来把外部服务接入 agent，例如浏览器、数据库、知识库或其他工具服务。MCP 的价值很明确：当任务需要外部系统能力时，它能把系统变成模型可调用的接口。它的成本也很硬：server 定义、工具 schema、使用说明都会占据上下文窗口。视频里给出的直觉是，一个 MCP server 可能消耗大量 token，多个 server 叠加后会挤压当前任务空间。

1.4 MCP、sub-agent 与 plan mode 的按需外接

Pi Agent 的默认策略可以概括为“按需外接”。MCP 在浏览器自动化、数据库读写、远程服务调用等场景很有用；边界在于，不要把所有可能用到的外部能力提前塞进每次

会话。更轻的做法是把某些外部工具包装成普通命令行工具，通过文档告诉 Pi Agent 何时调用。在这里，命令行工具承担的是 Unix 世界里“一个命令做好一件事”的组合接口。

sub-agent 是由主 agent 启动或调度的子 agent，适合把相对独立的检索、验证、实现任务分出去并行处理。视频里的判断更苛刻：许多 sub-agent 需求来自前期规划不足。若文档路径、目标文件、测试命令已经明确，主 agent 直接读取这些材料即可；若确实需要并行，可以让 Pi Agent 启动额外 Pi 实例，或通过扩展包补足调度能力。

plan mode 是内建计划模式，常见用途是让 agent 先写计划、再执行。Pi Agent 的极简选择是把计划还原成 Markdown 文件，例如 plan.md 或 todo.md。这样做的好处是计划变成普通文件：可以被版本控制、被人编辑、被 agent 重新读取，也可以自然进入代码审查和协作流程。计划的生命力主要来自可见、可改、可追踪；固定 UI 模式只是其中一种承载形态。

“

按需外接”的风险是用户需要承担更多工程判断。若任务涉及真实凭据、生产数据库、危险命令或大规模文件改写，轻量工具面不会自动替用户完成安全设计。此时应把权限、隔离、审计和回滚机制显式写入项目规则或扩展逻辑。

1.5 sandbox、tmux 与 Markdown 计划文件

sandbox 是安全隔离环境，用来限制命令和文件系统影响范围。可以把它理解为实验室防护罩：模型可以做实验，但实验材料、出口和危险动作受到控制。视频提到 Pi Agent 没有把频繁 permission block 作为主要安全体验。permission block 指命令执行前反复询问用户授权的拦截流程。它能降低误操作概率，却也会把安全问题变成来回确认。更稳的做法是结合 sandbox、项目规则和扩展里的可执行安全策略。

tmux 是 Unix 终端复用工具，可在一个终端里管理多个会话、窗口和后台命令。它对 agent 工作流的意义很实际：长时间运行的构建、服务器、测试监听器可以放在 tmux session 里；agent 或人类随后 attach 回这个 session，直接观察输出。与为每个后台任务开发专门监控界面相比，tmux 复用了终端生态已经成熟的可观测性。

这正是 Pi Agent 的极简主义：删除非必要结构，保留能够组合出复杂能力的核心机制。它和 Unix 哲学同频：普通文件保存计划，bash 执行命令，tmux 管理长期进程，sandbox 负责隔离，agent 负责把这些小工具组织起来。hackable 描述的就是这种工程可塑性：用户可以按需拆改、包装、扩展工具，避免被固定工作流锁住。

1.6 关键画面与结构图

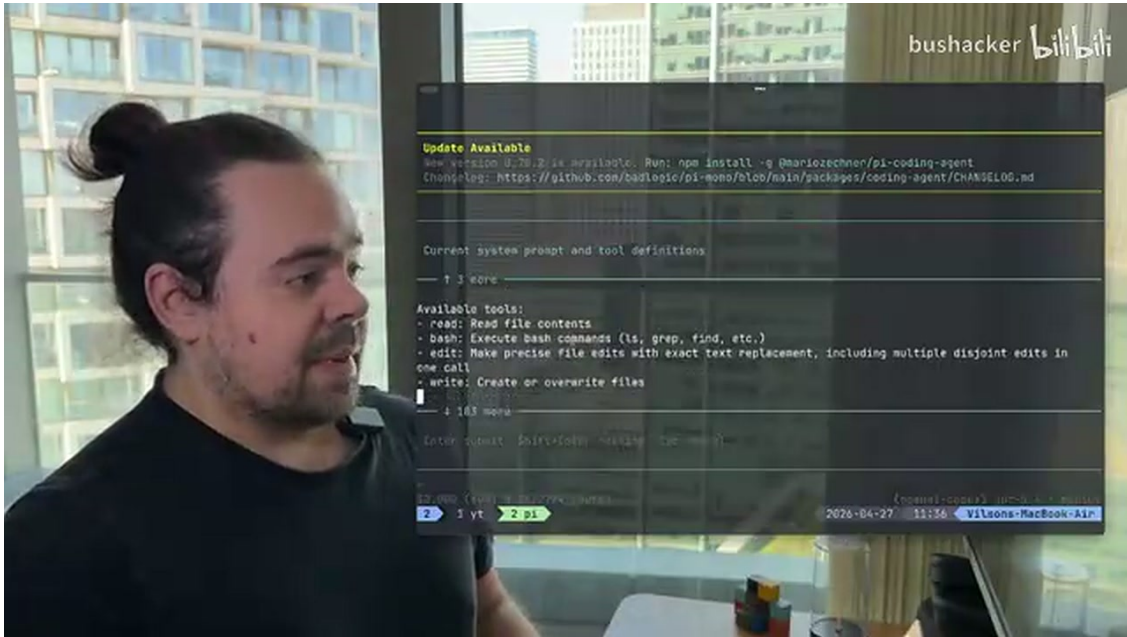
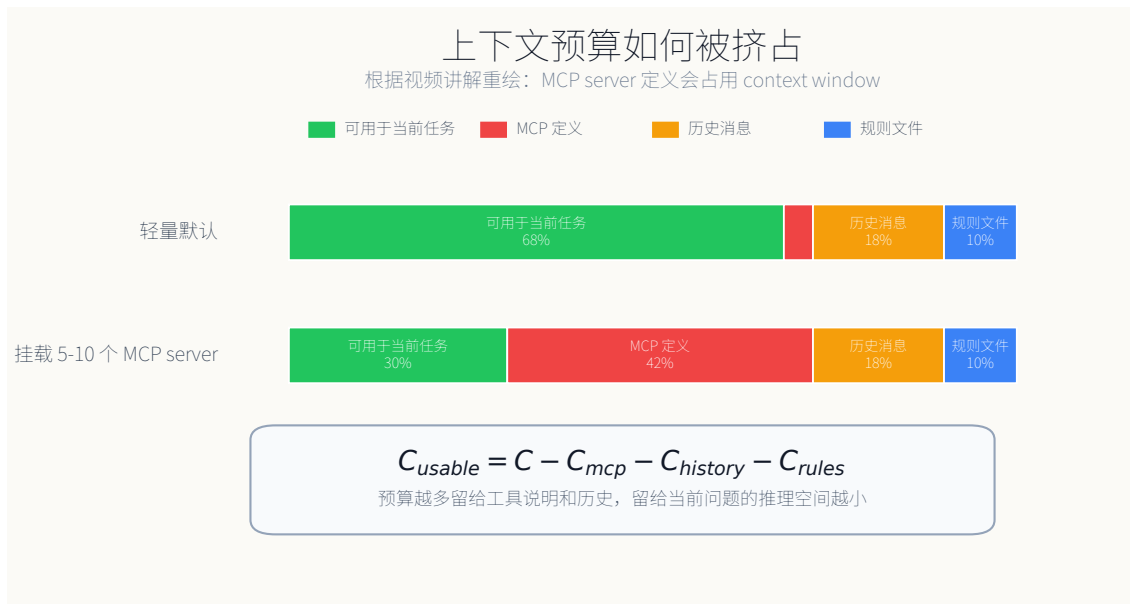


图 1: Pi Agent 默认工具与 system prompt 的原始视频截图。¹



图 2: Pi Agent 核心工具集示意图，根据视频讲解重绘。²

图 3：上下文预算被 MCP、历史消息和规则文件占用的示意图，根据视频讲解重绘。³

内建模式与普通工具组合的取舍
根据视频讲解重绘：sub-agent、permission block、plan mode 都按需外接

判断问题：是否值得把能力固化进 Pi 的默认工作面？
固化会增加规则、UI 状态和上下文成本；外接则保留 Unix 式可组合性。

需求	重型内建做法	轻量替代	工程含义
并行探索	sub-agent 常驻调度	开多个 Pi / tmux 会话	需要时横向扩展，默认界面保持薄
命令安全	permission block 流程	sandbox + 显式命令边界	把风险关在运行环境里，减少交互摩擦
计划管理	plan mode / todo mode	plan.md 普通文件	计划可编辑、可审计、可进版本库
功能扩展	预置复杂开关	extension / CLI 按需接入	把冷门能力放在边上，避免挤占主路径

极简主义的判断标准：默认路径只保留高频核心，低频复杂度从文件、CLI、tmux 和扩展接入。

图 4：内建模式与普通工具组合的取舍表，根据视频讲解重绘。⁴

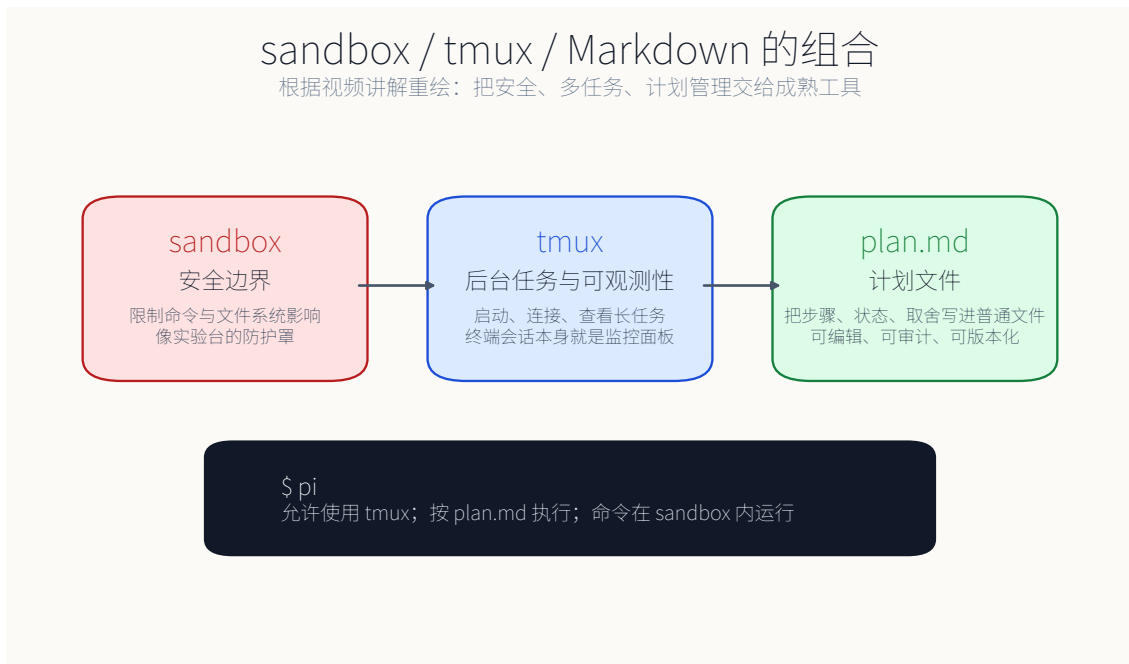


图 5: sandbox、tmux 与 Markdown 计划文件的组合关系，根据视频讲解重绘。⁵

1.7 本章小结

本章定义了 Pi Agent 的核心：它是一种以短 system prompt、少量基础工具和可扩展机制为中心的编码 agent。它把 Claude Code 这类更完整座舱作为对照，选择把默认能力面压薄，把复杂 workflow 交给模型已学会的编码循环、普通文件、命令行工具、sandbox 和 tmux。

这套设计的取舍很清楚：上下文窗口是稀缺资源，MCP、sub-agent、plan mode 和 permission block 都有价值，但默认装满会挤压当前任务证据。Pi Agent 的立场是先保持核心轻量，再在确有需求时外接能力。极简主义在这里落到一套工作方法：由 token 预算、Unix 工具链和可 hack 工程边界共同支撑。

2 安装、Provider 与模型自由

本章要解决的问题是：用户如何启动 Pi Agent，并把它连接到订阅模型、OpenRouter、本地模型和高频交互命令。上一章解释了 Pi Agent 为什么保持轻量；这一章进入操作层，说明轻量核心如何通过 provider、API key、模型切换和配置目录获得模型自由。

2.1 安装与首次启动

Pi Agent 的安装路径很短：复制安装命令，安装完成后在终端输入 `pi` 启动。首次进入时，界面会显示版本号，并提示可以用快捷键查看更多命令。这里的终端入口很重要，因为 Pi Agent 的主要使用方式是 CLI，即 Command Line Interface，命令行界面。CLI 像程序员的控制台：输入命令、观察输出、把小工具接成 workflow。

安装的关键是入口清楚、反馈明确。一旦 `pi` 能启动，用户接下来最该配置的是模型访问方式。Pi Agent 自身提供的是工作面和工具循环，实际推理能力来自背后的模型服务。换句话说，安装只是把外壳放到本机；provider 配置才决定这把工具连接哪台引擎。

首

次启动后建议先完成两件事：确认 `pi` 命令可用；确认当前会话知道要使用哪个 provider 和哪个 model。前者是工具入口，后者是推理来源。缺少任意一项，agent workflow 都会停在半路。

2.2 Provider 解耦：订阅、OpenRouter 与 API key

Provider 是模型服务提供方，例如 OpenAI/Codex 订阅、OpenRouter，或用户自己运行的本地模型服务。Pi Agent 的一个核心自由度是 provider 解耦：同一个工作界面可以接不同模型来源，用户不必把所有任务绑定到单一服务商。

订阅登录是一条路径。用户在 Pi Agent 里执行登录命令，选择 ChatGPT Plus、Pro 或 Codex 订阅一类入口，浏览器完成授权后，Pi Agent 就能通过订阅访问相应模型。这个路径适合已经有订阅的用户，优点是上手快、凭据管理相对集中。

另一条路径是 API key。API 是 Application Programming Interface，即应用程序接口；API key 是访问模型服务的凭据，作用类似一把能计费、能鉴权的钥匙。它必须被视作敏感配置，避免写入公开仓库、截图或共享文档。视频中示范了 OpenRouter：OpenRouter 是多模型 API 聚合服务，用户可以通过一个 key 访问大量模型。典型流程是创建 OpenRouter 账号、充值额度、进入 API keys 页面生成 key，再把它配置为环境变量或写入认证配置。

A

PI key 泄露会带来真实费用和账号风险。教学笔记中可以展示变量名和配置位置，但不应展示真实 key。若需要在 shell 中配置，优先使用环境变量或本机私有配置文件，并确认这些文件不会被版本控制提交。

这里还会遇到 URL 和 JSON。URL 是 Uniform Resource Locator，统一资源定位符，用来描述服务地址；JSON 是 JavaScript Object Notation，常用于配置 provider、模型、endpoint 和参数。它们在后续本地模型配置中会频繁出现。

2.3 模型切换、fork 与本地模型

模型切换指在同一 Pi Agent 工作面中改变当前 provider 或 model。视频示例中，用户可以进入 model 命令，选择 OpenRouter，再从大量模型里挑一个；状态栏会显示当前 provider 和模型名。随后也可以切回 Codex 订阅模型。对程序员而言，它的价值在实验方法：同一任务可以交给更便宜、更快、更强推理或更适合代码的模型。

模型切换和会话分支结合时尤其有用。后文会系统解释 fork 机制；本章只保留操作直觉：同一个任务可以沿不同模型路径继续探索。一个实际策略是：先用成本低、速度快的模型做初稿，再切到更强模型检查边界条件；或在分支里比较两个模型对同一 bug 的修复方案。

本地模型是在本机或局域网 endpoint 运行，并通过配置文件接入 Pi Agent 的模型。endpoint 是服务入口地址，常见形式是一个本机 URL。随着开源代码模型增强，本地模型会越来越常见：它能降低外部调用依赖，也能让敏感代码留在局域环境中。配置时通常要写清 endpoint、模型名和 context window。context window 沿用上一章定义，表示模型一次推理能接收的 token 空间。

把

provider、model 和会话历史区分开会减少混乱：provider 是“模型从哪里来”，model 是“具体用哪一个模型”，会话历史记录任务如何走到当前状态。模型可以切换，provider 可以更换，历史记录应保持可追踪。

2.4 常用快捷键与命令入口

Pi Agent 的高频交互依赖快捷键和命令。快捷键是键盘上的快速操作入口，命令是 CLI 里的显式动作入口。两者解决的都是同一个问题：用户不应为退出、取消、换行、清空 prompt 这类动作离开键盘思考太久。

常见操作可以整理成几类。退出 Pi Agent 时使用 **Ctrl+C**；如果模型正在生成或命令正在运行，**Ctrl+C** 也可用于取消当前动作。输入长 prompt 时，**Shift+Enter** 用于换行，避免把尚未写完的多行指令提前发送。若当前 prompt 写乱了，可以用取消操作清空输入。界面还提供命令帮助入口，用于查看更多命令和快捷键。这里的设计重点是肌肉记忆：退出、取消、换行、清空、查帮助这些动作越稳定，用户越能把注意力留给需求拆解和代码判断。

这些快捷键看似琐碎，实际影响很大。编码 agent 的交互密度高，用户会频繁补充约束、取消错误方向、重新组织长指令。一个稳定的键盘模型能降低认知切换：用户的大脑继续留在任务上，手指完成界面控制。

2.5 配置文件夹的总体地图

配置目录是 Pi Agent 保存全局设置、认证信息、模型定义、扩展、主题、会话等文件的集中位置。视频里提到，用户安装或配置的内容会放在主目录下的 `.agent` 文件夹中。可以把它理解为 Pi Agent 的“用户态控制室”：模型怎么接、认证怎么存、会话怎么留、扩展怎么加载，都能在这里找到痕迹。

这一目录通常会容纳几类对象：authentication 相关配置用于记录 provider 登录或 key；models 相关 JSON 用于定义本地模型或额外模型；settings JSON 保存偏好设置；会话历史目录保存交互记录；扩展相关目录、能力包目录、主题目录等承载后续生态。这里的 JSON 文件相当于机器可读的说明书：键名给出配置项，值给出 endpoint、模型名、上下文窗口或开关状态。相关机制会在后续章节展开，本章只标出它们在文件系统地图里的位置。

全局 system prompt 也可以通过配置目录里的 `system.md` 覆盖默认行为；项目级规则文件的具体机制留到后文展开。本章只需要抓住分工：全局文件描述用户希望 Pi Agent 长期遵守的个人偏好，项目文件描述当前仓库的局部事实、测试命令和协作规则。把全局与项目分开，能避免一个项目的特殊规则污染所有项目。

2.6 关键画面与结构图

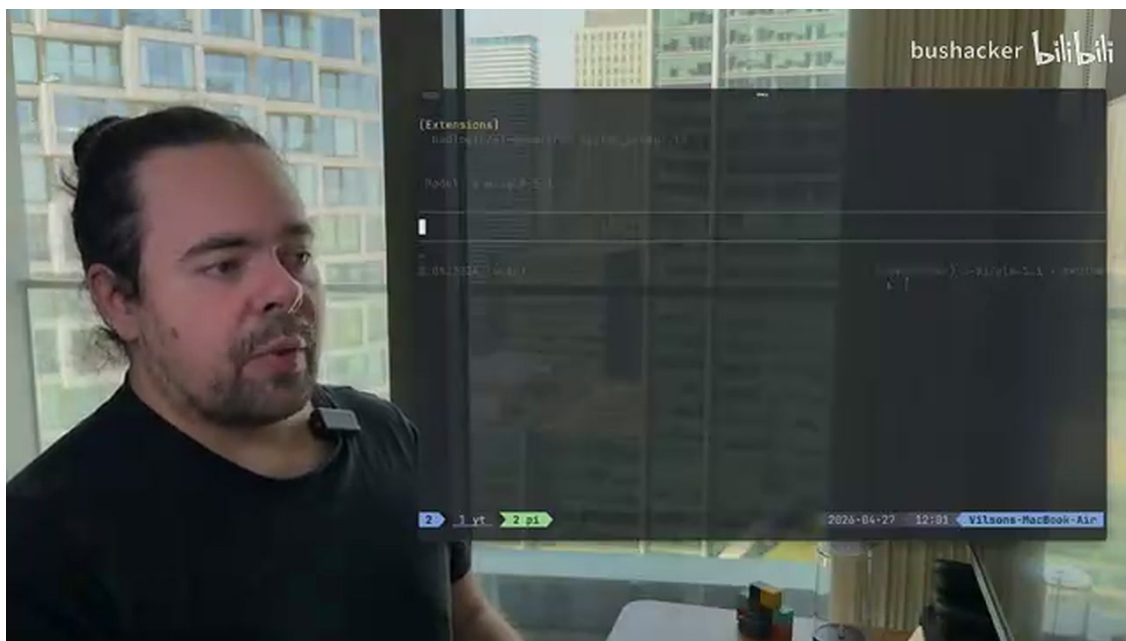


图 6: Provider 与模型切换状态栏的原始视频截图。⁶



图 7：安装与首次启动命令框，根据视频讲解重绘。⁷

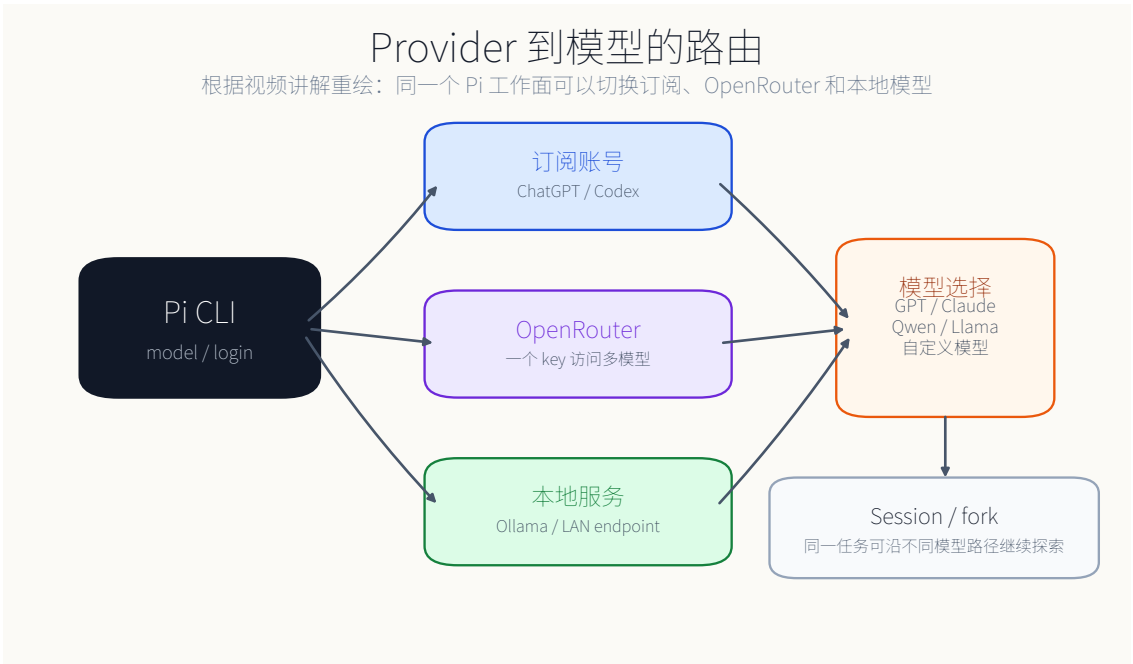


图 8：Provider 到模型再到会话的路由图，根据视频讲解重绘。⁸



图 9：本地模型配置字段表，根据视频讲解重绘。⁹

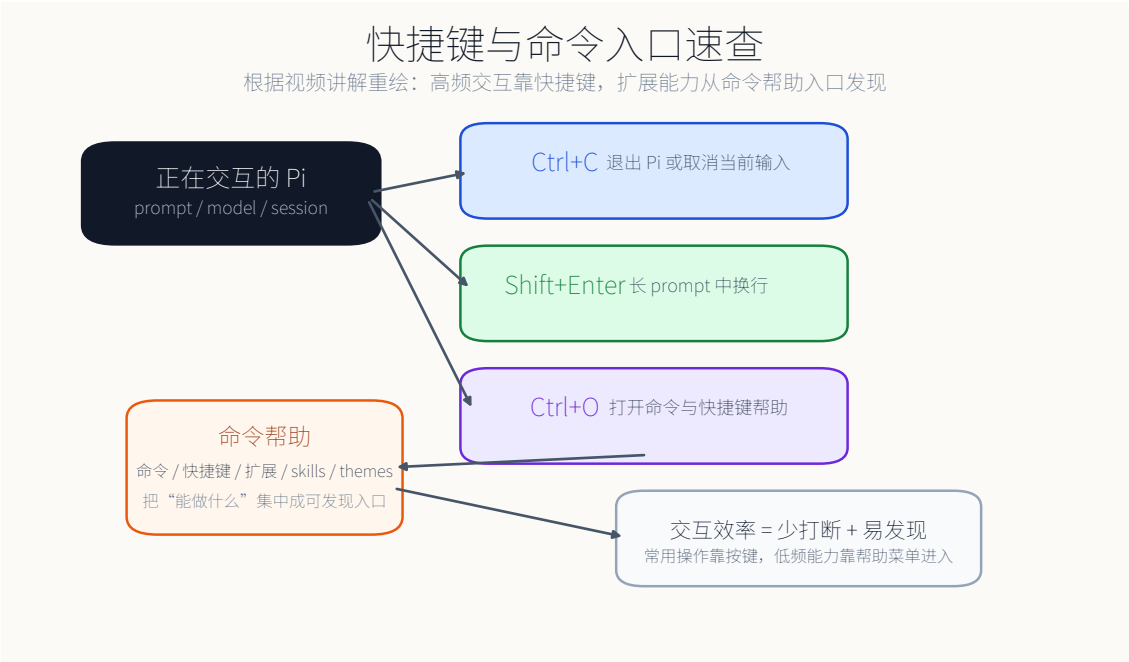


图 10：快捷键与命令入口速查图，根据视频讲解重绘。¹⁰

2.7 本章小结

本章把 Pi Agent 的启动和模型接入串起来：先安装并运行 pi，再配置 provider，随后通过订阅登录、OpenRouter API key 或本地模型 endpoint 获得模型能力。Provider 解耦让 Pi Agent 像模型界的瑞士军刀，用户可以按成本、速度、隐私和推理能力选择合适模型。

操作层面的重点也很硬：API key 是敏感凭据，模型切换要看清当前 provider 和 model，快捷键负责降低高频交互成本，配置目录负责把认证、模型、会话和扩展组织成可检查的文件系统结构。掌握这些后，Pi Agent 就从一个刚装好的 CLI，变成可按个人工作流调校的编码工具。

3 Session 树：可恢复、可分叉、可审计的对话历史

这一节要解决的问题很具体：当一次 agent 对话结束以后，用户到底留下了什么？如果答案只是一屏聊天记录，那它很快就会变成一次性的消耗品；如果答案是一组可恢复、可分叉、可导出的结构化历史，它就变成了工程资产。Pi Agent 在这里给出的思路是把对话当作 session 树来管理，让探索过程像 Git 分支、终端日志和实验记录一样，可以回看、复用和审计。

3.1 Session 文件为何像工程日志

Session 指一段 agent 对话及其事件历史。它包含用户消息、助手回复、工具调用、模型切换，以及会话运行过程中产生的其他关键事件。更准确地说，session 属于一段工作过程的结构化轨迹；它像实验室记录本，记录“当时给了什么输入、模型怎样回应、期间发生了哪些状态变化”。

Pi Agent 将这些历史保存成 JSONL 文件。JSONL 是 JSON Lines 的缩写，含义是“每一行都是一个独立 JSON 对象”。和一个庞大的 JSON 数组相比，JSONL 更适合持续追加：新的消息到来时，只需要在文件末尾写入一行。它也适合被命令行工具流式读取，因为读者可以逐行处理，不必先把完整文件加载进内存。可以把它理解为数据库日志的轻量版本：一行就是一个事件，一串行就还原出整个 session。

这种记录方式有两层工程价值。第一层是可检查性：用户可以离开 Pi Agent，用编辑器、脚本、日志分析工具直接查看会话文件。第二层是可迁移性：JSON/JSONL 属于通用格式，后续的导出、统计、审计和自动化都可以围绕它展开。字幕里提到模型切换也会被记录下来，这一点很关键，因为模型选择会影响回答质量、成本和行为风格；把它写入历史，等于给日后的问题定位留下证据链。

3.2 tree：把对话看成分支结构

tree 是浏览 session 分支结构的主要命令。它把连续聊天从“直线时间轴”转换为“树”。树的根部是会话起点，节点是用户消息或关键交互，边表示上下文继承关系。用户在某个节点继续提问，就会沿着当前路径向下生长；用户从旧节点另开路线，就会长出一条新枝。

这和 Git 分支有一个直观类比：同一个代码库可以从某个提交点分出不同分支，同一个 session 也可以从某条历史消息分出不同探索路径。区别在于，Git 管代码状态，session 树管认知状态。前者关心文件如何演化，后者关心上下文如何演化。对于程序员来说，这个模型很自然：一次调试可能有“继续追日志”“改测试复现”“直接读源码”三条路线，tree 让这些路线同时存在，并且能被看见。

需要强调的是，树结构让上下文变得可选择。用户可以沿着包含大量背景的老分支继续推进，也可以回到较早节点，选择较轻的历史开始探索。这里的边界在于：更丰富的历史通常带来更多线索，也会带来更多 token 消耗和旧假设干扰。会话树把这个取舍显性化，用户可以按任务选择上下文密度。

3.3 fork：从历史节点另开实验路径

fork 的作用是从一个历史节点创建新会话。它继承该节点之前的必要上下文，然后从那里开始新的路径。若说 tree 负责“看见分支”，fork 就负责“制造分支”。这让用户能够把一次对话变成多组实验，而不必把所有尝试塞进同一条聊天流里。

视频里的例子是从旧消息处创建全新对话，新会话只带有那一段历史所需的上下文。这个细节非常实用：许多 agent 任务失败并非模型完全无能，常见原因是上下文里混入了过多旧讨论、旧约束和过期假设。对程序员而言，这就像在一个污染严重的 REPL 环境里继续试验，变量名、状态和副作用都可能误导下一步。通过 fork，用户可以回到干净一些的历史节点，把新的实验从那里展开。

fork 尤其适合三类场景：第一，同一需求存在多种实现路线，需要并行比较；第二，前一轮讨论已经偏航，用户想保留有用背景并切掉后续噪声；第三，需要用不同模型或不同提示策略处理同一问题。它把“试错”从一次性赌博改成可回溯实验，失败路径也有价值，因为它们记录了哪些想法被排除。

3.4 恢复、分享、导出与统计

会话树要变成工作流，还需要一组入口命令。字幕中提到的 continue 类命令用于回到上一个 session，resume 类命令用于从列表中选择特定旧 session。前者像“继续刚才的终端任务”，后者像“从项目档案里挑一份旧实验记录”。它们共同解决一个现实问题：程序员的工作经常跨天、跨分支、跨任务切换，重要上下文不能只停留在当前终端窗口里。

share 用于把 session 变成协作材料。配置相应凭据后，它可以生成可分享链接，让同事看到同一段探索历史。这里的价值不只在“把答案发出去”，还在于把答案背后的路径发出去：对方能看到哪些指令被尝试过、agent 怎样理解问题、哪些分支曾经出现。对于代码审查、结对调试和交接任务，这比复制最后一段回复可靠得多。

`export` 则偏向归档和审计。它可以把 `session` 导出为 HTML 等更适合阅读和转发的格式，并保留树状消息结构。应区分 `share` 与 `export`：前者偏协作流转，后者偏离线留存。前者像把项目链接发给队友，后者像把会议纪要归档到知识库。

`session` 命令负责统计当前会话的 token 消耗与成本。token 是模型处理文本的计量单位，成本统计把“上下文用量”从感觉变成数字。一个轻量的成本模型可以写成：

$$\text{cost} = \sum_i \left(\frac{T_i^{\text{in}}}{10^6} P_i^{\text{in}} + \frac{T_i^{\text{out}}}{10^6} P_i^{\text{out}} \right)$$

其中 T_i^{in} 与 T_i^{out} 分别表示第 i 次调用的输入和输出 token 数， P_i^{in} 与 P_i^{out} 表示对应的百万 token 单价。这个公式不追求覆盖所有计费细节，它的教学意义在于提醒用户：长历史、频繁工具调用和大段输出都会进入账单。Pi Agent 把统计放进 `session` 命令里，等于让上下文成本变成可观测资源。

3.5 会话树带来的协作价值

Session 树的真正价值在于把“对话”提升为“工程过程”。普通聊天记录只回答“最后说了什么”，`session` 树还能回答“怎么走到这里”。当一个任务需要交接时，接手者可以从导出的树里看到思路演化；当一个方案需要复盘时，团队可以回到分叉点比较不同路线；当一次昂贵调用产生账单疑问时，统计信息和 JSONL 历史可以帮助定位成本来源。

这也解释了为什么 Pi Agent 的极简取向并没有削弱可控性。它选择把关键状态落到文件和命令里：历史由 JSONL 承载，结构由 `tree` 呈现，实验由 `fork` 打开，协作由 `share` 连接，归档由 `export` 完成，成本由 `session` 量化。每个命令都很小，组合起来却覆盖了从个人探索到团队审计的完整链条。

3.6 关键画面与结构图

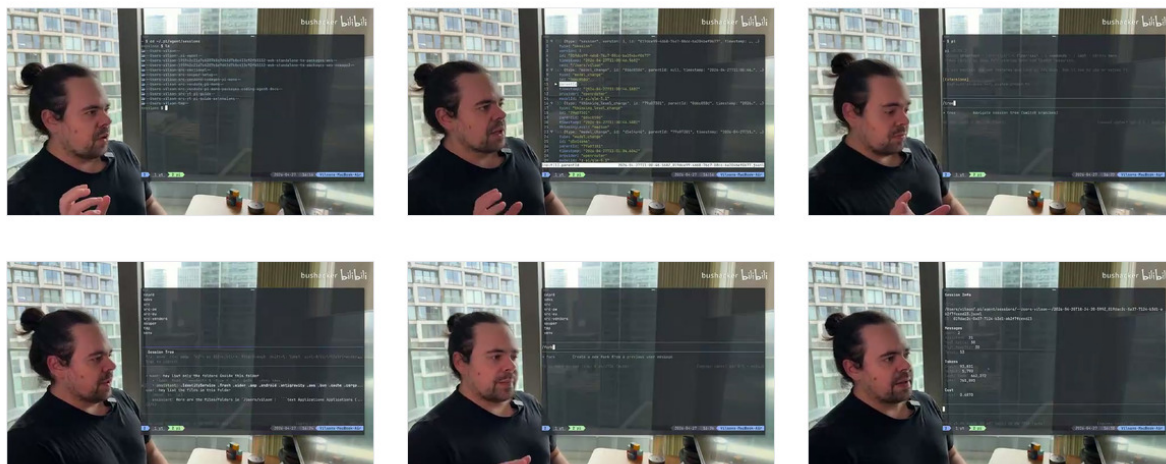


图 11: Section 3 来源帧组：视频依次展示 session JSONL、tree/fork 分支和 session 统计；因源视频仅 640×360，小字仅作 provenance，教学解释以本节重绘图为准。¹¹

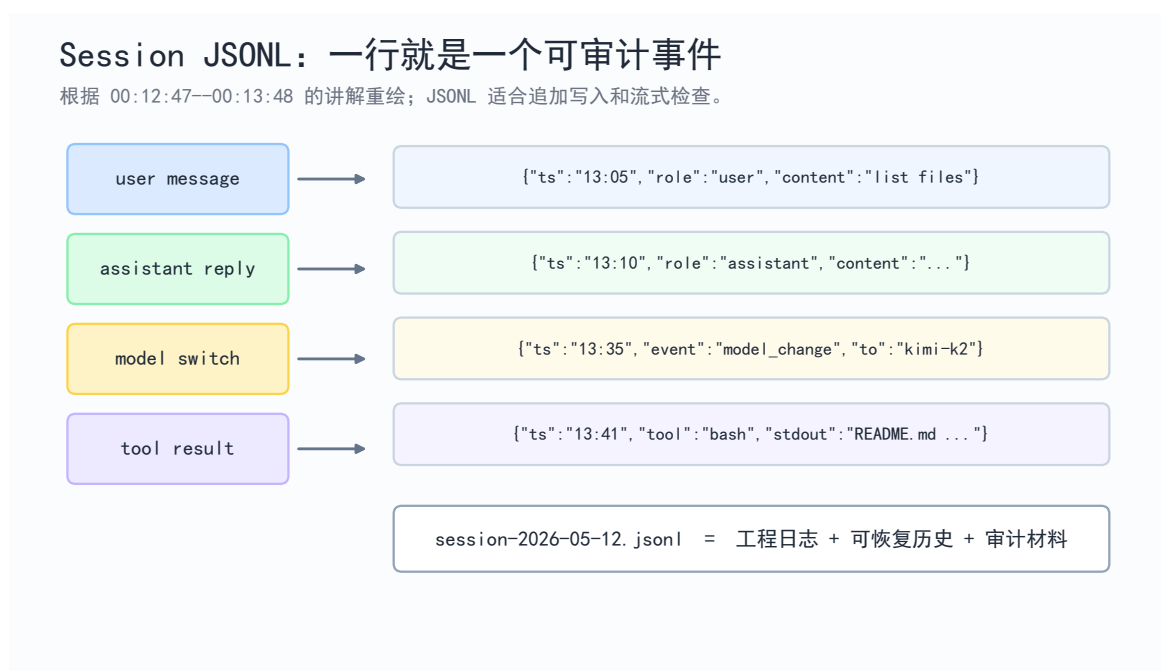


图 12: Session JSONL 结构示意图（根据视频讲解重绘）：每条消息、模型切换和工具结果都以一行 JSON 进入可审计日志。¹²

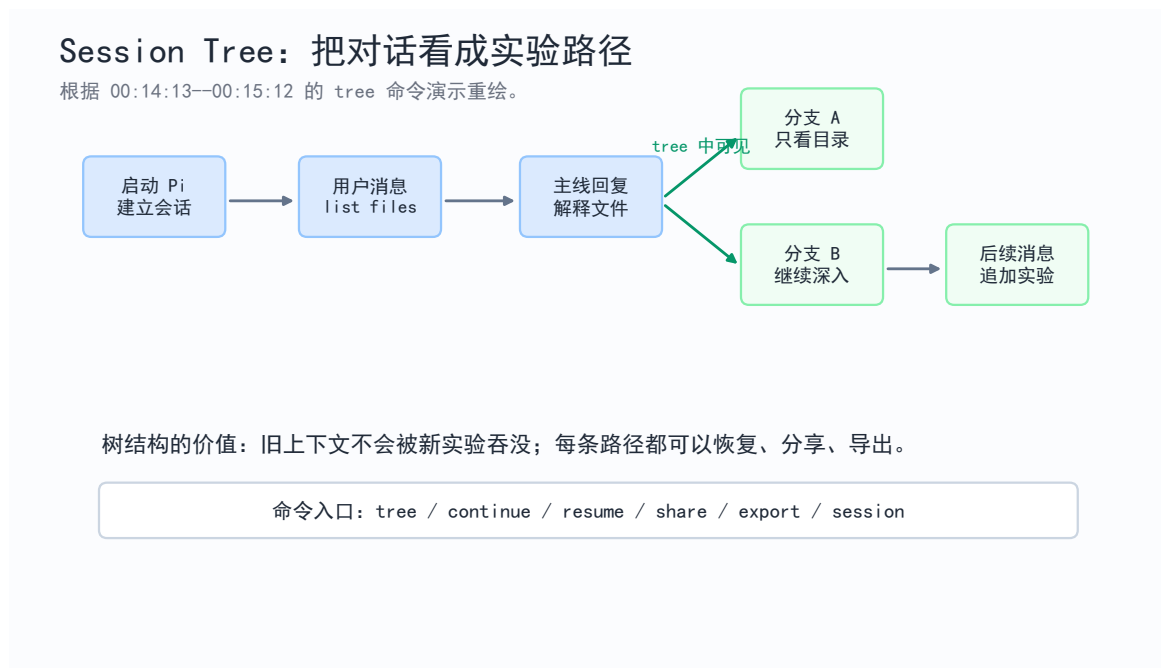


图 13: Session tree 图（根据视频讲解重绘）：tree 命令把一次对话展开为可恢复、可比较的实验路径。¹³

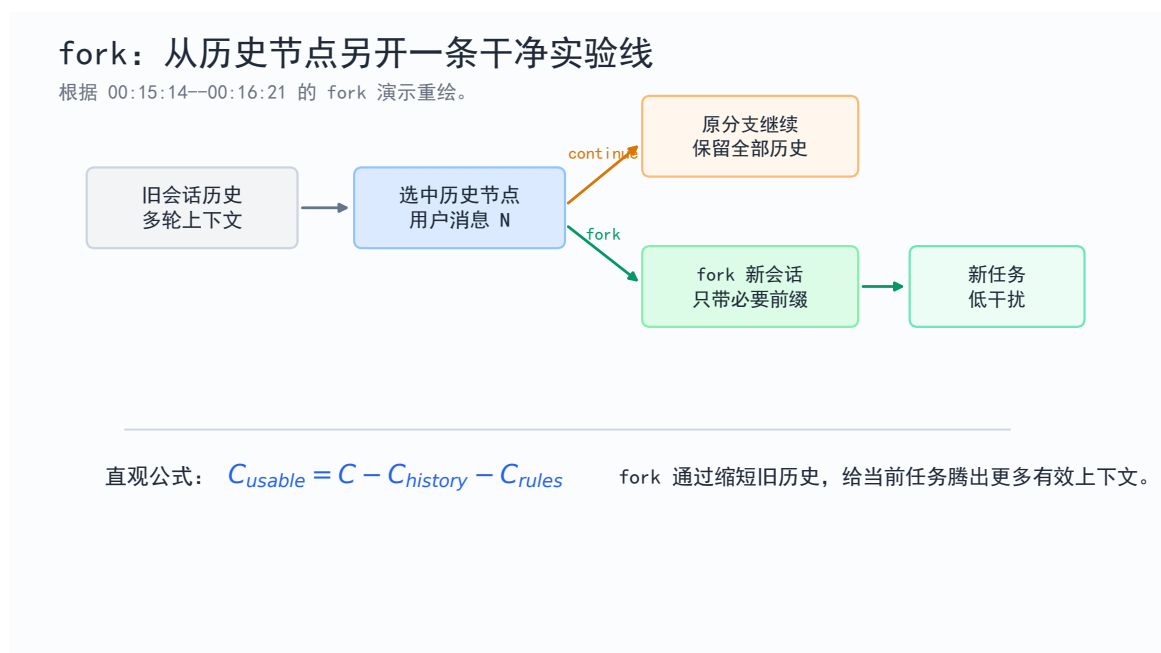


图 14: fork 分支图（根据视频讲解重绘）：从历史节点另开新会话，以更少旧上下文承载新任务。¹⁴

session 统计：把上下文消耗变成可观测资源		
根据 00:16:22--00:17:52 的 share/export/session 统计讲解重绘。		
观测项	示例值	工程意义
input tokens	612,000	历史和规则占用的阅读量
output tokens	91,000	模型生成的回答和代码量
total tokens	703,000+	衡量会话体量与压缩时机
estimated cost	\$0.58	把探索成本纳入决策

配套命令：share 负责协作链接，export 负责 HTML 审计材料，session 负责 token 与费用统计。

图 15: token/cost 观测小表（根据视频讲解重绘）：session 统计把上下文消耗和费用暴露为可管理指标。¹⁵

3.7 本章小结

本章把 Pi Agent 的 session 机制理解为一套“对话版本控制”。Session 是对话和事件历史，JSONL 是可追加、可流式读取的日志格式；tree 让历史呈现为分支结构，fork 让用户从旧节点开启新实验；continue/resume 负责回到旧会话，share/export 负责协作与归档，session 负责 token 与成本统计。它给程序员的启发很直接：agent 工作的产物不应只是一段回答，还应包括可恢复、可比较、可审计的探索路径。

4 Context Engineering: 把该进上下文的东西放进去

上一章说明了 session 历史如何被保存、分叉和审计。本章进入更靠近 agent 行为源头的问题：模型在某一轮推理时到底看见了什么？Context engineering 讨论的就是这件事。它关心哪些信息进入上下文窗口，哪些信息被留在外部文件里，哪些规则应该成为长期约束，哪些提示只适合一次性任务。对于程序员来说，这相当于给 agent 设计“工作台”：桌面上摆什么工具、贴什么规范、打开哪些文件，都会影响它下一步怎么做。

4.1 上下文工程的基本问题

Context engineering 指主动设计上下文输入的工程活动。它决定模型看见什么、忽略什么、依据什么行动。这里的上下文既包括用户当前输入，也包括项目规则、系统

提示、历史摘要、工具说明、skills、prompt 模板和被检索进来的文件片段。模型没有人类那种稳定的长期工作记忆；它每次推理主要依赖当前窗口中的 token。因此，把信息放进上下文窗口，近似于把材料递到它眼前。

上下文工程的难点在于空间有限。可以用一个轻量预算式理解：

$$C_{usable} = C - C_{mcp} - C_{history} - C_{rules}$$

其中 C 是总上下文预算， C_{mcp} 是 MCP 相关描述占用， $C_{history}$ 是历史消息占用， C_{rules} 是规则文件和能力说明占用， C_{usable} 是留给当前任务材料的空间。这个式子只用于直观解释：规则写得越长，历史压得越多，当前代码和需求可用的空间就越少。真正的工程判断在于取舍，而取舍要围绕任务目标展开。

Context engineering 的目标可以概括为三点：让必要约束稳定出现，让当前任务材料足够具体，让过期噪声尽量少进入推理。规则像路标，材料像地图，历史像旅行记录。路标缺失时 agent 容易违反项目约定；地图太少时它只能猜；旅行记录太厚时，它又会被旧路径拖住。

4.2 AGENTS.md：项目级说明书

AGENTS.md 是项目级 agent 指南文件。它通常放在项目根目录，也可以放在子目录或用户级配置目录，用来承载代码质量要求、测试命令、架构说明、重要文件位置、协作约定和常用工作流。可以把它理解成写给 agent 的“项目入职手册”：新人进项目需要知道怎么跑测试、哪里是核心模块、哪些命令有副作用，agent 也一样需要这些边界。

一个有效的 AGENTS.md 应该偏向可执行信息，而非愿景口号。比如“修改解析器后运行 `npm test -- parser`”比“保证高质量”更有用；“数据库迁移只允许新增迁移文件”比“注意数据安全”更能约束行为。它还适合记录团队约定，例如 PR 描述格式、禁止改动的生成文件、测试慢路径、需要人工确认的部署命令。

应区分项目级规则和一次性任务提示。项目级规则具有长期稳定性，适合写进 AGENTS.md；一次性目标会随着当前任务结束而失效，适合留在用户 prompt 或临时计划里。把短期目标写进长期规则，会让后续 session 背上过期约束；把长期规则只写在聊天里，又会导致下一次任务遗忘。上下文工程首先是位置工程：同一句话放在不同入口，生命周期完全不同。

4.3 system.md 与 append-system.md：角色和规则注入

system.md 是覆盖默认 system prompt 的文件。system prompt 是写给模型的顶层角色和行为指令，前文已经说明 Pi Agent 默认保持很短；system.md 的意义在于让用户

构造一个专门角色。视频中的示例把 agent 设为代码审查员，只提供读取能力，不提供写入能力。结果是当用户要求它创建脚本时，它会按角色边界拒绝写入任务。这说明系统层规则会改变 agent 对自身职责和工具权限的理解。

`append-system.md` 则是在默认 system prompt 后追加规则。它保留原本的基础行为，再叠加额外约束。字幕中的例子是追加“先创建单元测试、持续迭代直到测试通过”的规则，于是 agent 在写斐波那契脚本时先生成测试，再运行测试并修正。边界在于：`system.md` 适合重塑角色，`append-system.md` 适合加一层流程纪律。前者像换一份岗位说明，后者像给现岗位贴一条操作规程。

CLI 参数也可以承担临时注入。它适合一次性角色或短期规则，比如本次只做 review、本次输出 JSON、本次优先补测试。若某条规则会反复使用，应沉淀成文件；若某条规则只服务当前命令，放在 CLI 参数或用户 prompt 中更清爽。直说就是：把所有东西都塞进 system 层会让 agent 变得沉重，甚至自相矛盾；上下文工程要求用户承认规则也有维护成本。

4.4 Compaction 的收益与损耗

Compaction 指对长会话进行摘要压缩，以延长上下文可用时间。Pi Agent 支持自动压缩，也可以通过 slash compact 强制触发。它的收益很直接：历史消息太长时，压缩能释放空间，让后续任务继续进行。对于跨天工作、长调试链路和大型重构，compaction 像把一摞现场笔记整理成阶段纪要。

但 compaction 是有损压缩。摘要会保留它认为重要的目标、结论和状态，也会丢掉细节、措辞、失败尝试和某些边缘约束。这里可以类比图片压缩：缩略图足以帮人认出画面，却无法还原每个像素。对 agent 来说，被丢掉的像素可能正好是下一轮排错需要的异常日志或命令输出。因此，压缩后的上下文适合继续主线，完整追责仍要回到原始 session JSONL。

这也把第三章和本章连起来：session JSONL 是历史底座，compaction 是当前窗口的减负手段。二者承担不同任务。前者保真，便于审计和恢复；后者省空间，便于继续推理。成熟的工作流会同时使用二者：让 compact summary 承载“下一步该怎么走”，让 JSONL 历史承载“过去到底发生过什么”。

4.5 Skills 与 Prompt 模板的版本化

Skills 是能力包，通常由使用说明、规则约束和可调用 CLI 工具组成。它们告诉 agent 在某类任务中应该怎样行动，必要时还提供脚本或命令作为执行支点。和普通提示词相比，skill 更像“带工具的操作手册”：它既描述方法，也约定入口，还可能包含可复用代码。

Prompt 模板是可参数化复用的提示词文件，适合重复任务。例如 review 模板可以预留一个“关注点”参数；调用时填入“测试覆盖率”，模板就把这个关注点嵌入固定审查框架。模板的价值在于稳定输出结构，减少每次手写 prompt 的漂移。它适合周报、代码审查、迁移计划、测试策略、发布说明等重复性强的任务。

Skills 和 prompt 模板都应该纳入版本控制。原因很朴素：它们会改变 agent 行为。若一条测试规则、一个脚本路径或一个审查模板悄悄变化，团队需要知道是谁改的、为什么改、影响哪些任务。版本化还带来共享能力：一个项目沉淀下来的 skill 可以被复制到相邻项目，一个成熟的 prompt 模板可以变成团队规范的一部分。

还有一个容易忽略的点：扩展也可以动态改变上下文。也就是说，上下文工程不只发生在启动时的文件加载阶段，也可能发生在运行时。后续章节会讨论扩展机制；本章只需抓住原则：任何能把信息、规则或工具说明放进模型窗口的入口，都会参与塑造 agent 行为。

4.6 关键画面与结构图

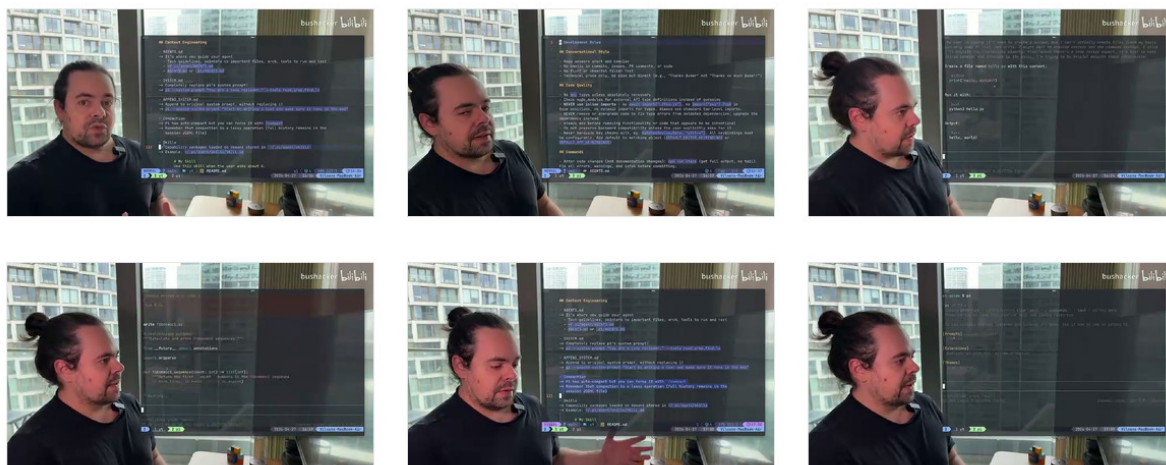


图 16: Section 4 来源帧组：视频依次展示 AGENTS.md、system.md、append-system.md、compaction 和 prompts；因源视频仅 640×360，小字仅作 provenance，教学解释以本节重绘图为准。¹⁶

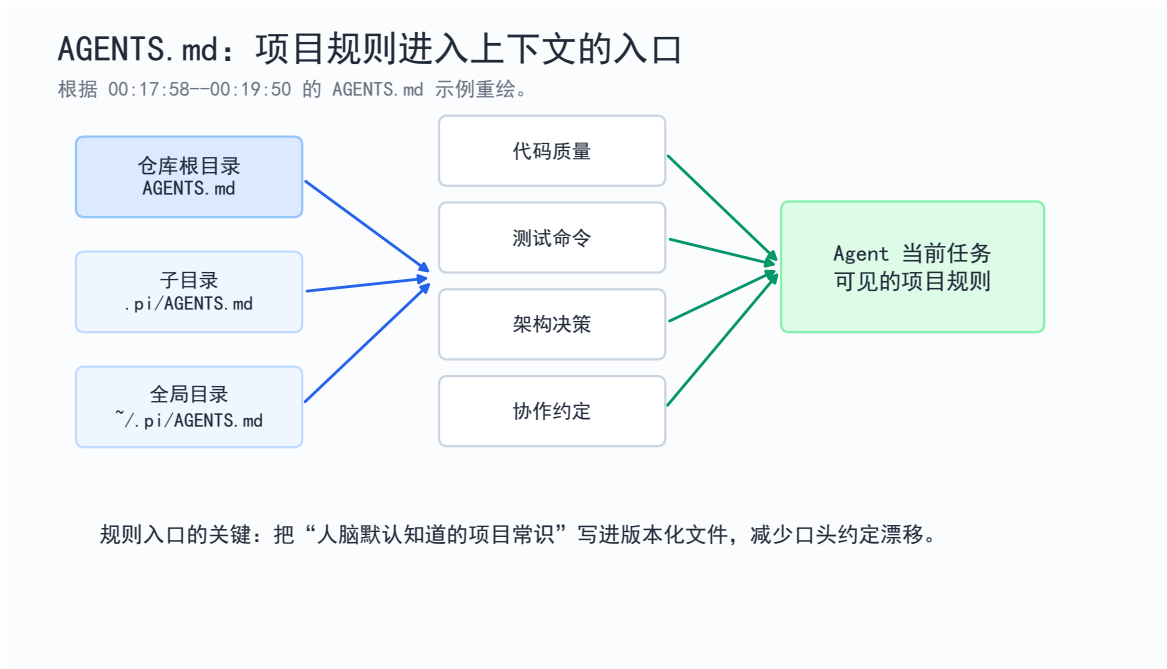


图 17: AGENTS.md 项目规则入口图（根据视频讲解重绘）：项目规则、测试命令和架构约定通过文件进入 agent 上下文。¹⁷

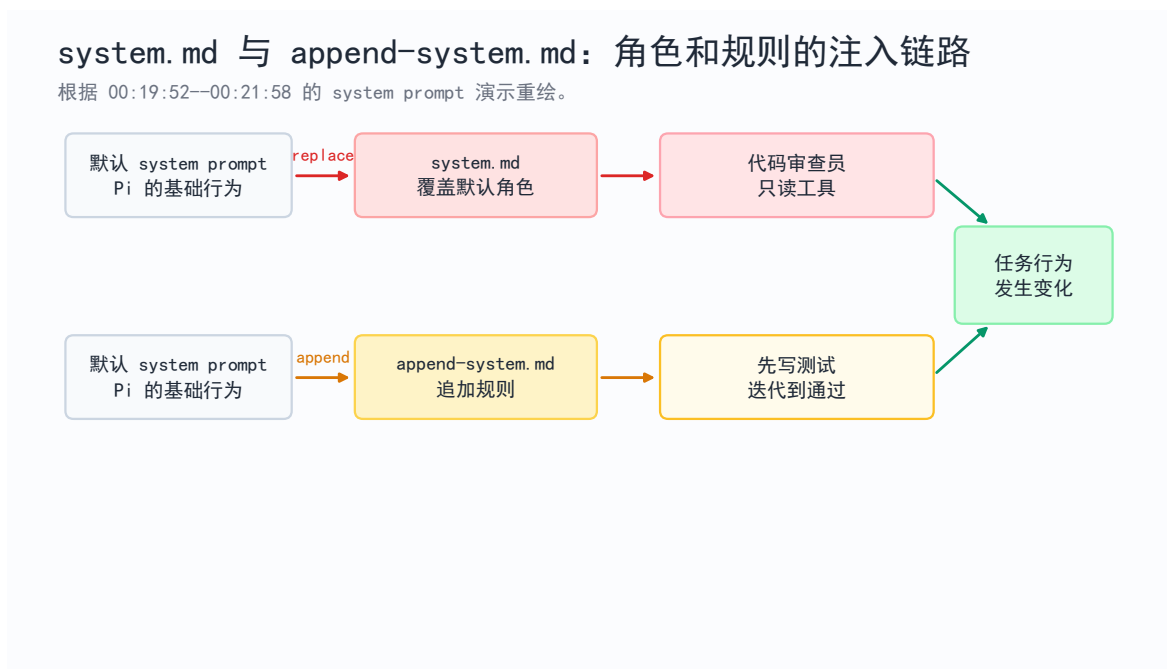


图 18: system.md / append-system.md 注入链路图（根据视频讲解重绘）：覆盖与追加分别改变角色和任务规则。¹⁸

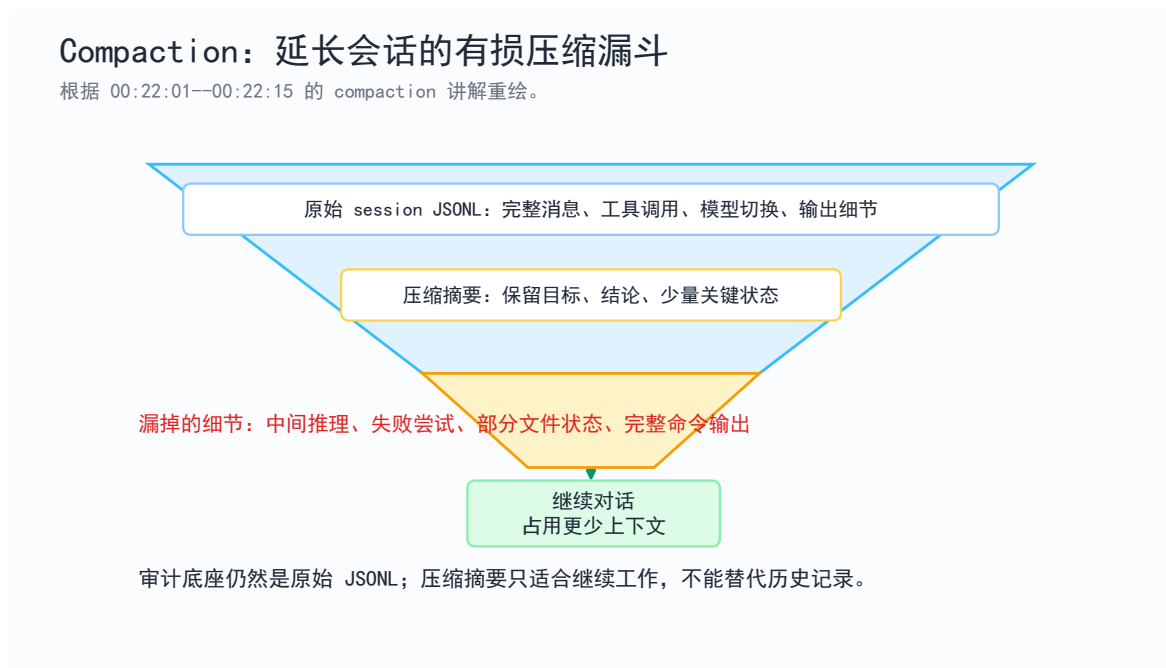


图 19: compaction 有损压缩漏斗图 (根据视频讲解重绘): 摘要能延长会话, 但完整审计仍依赖原始 session JSONL。¹⁹

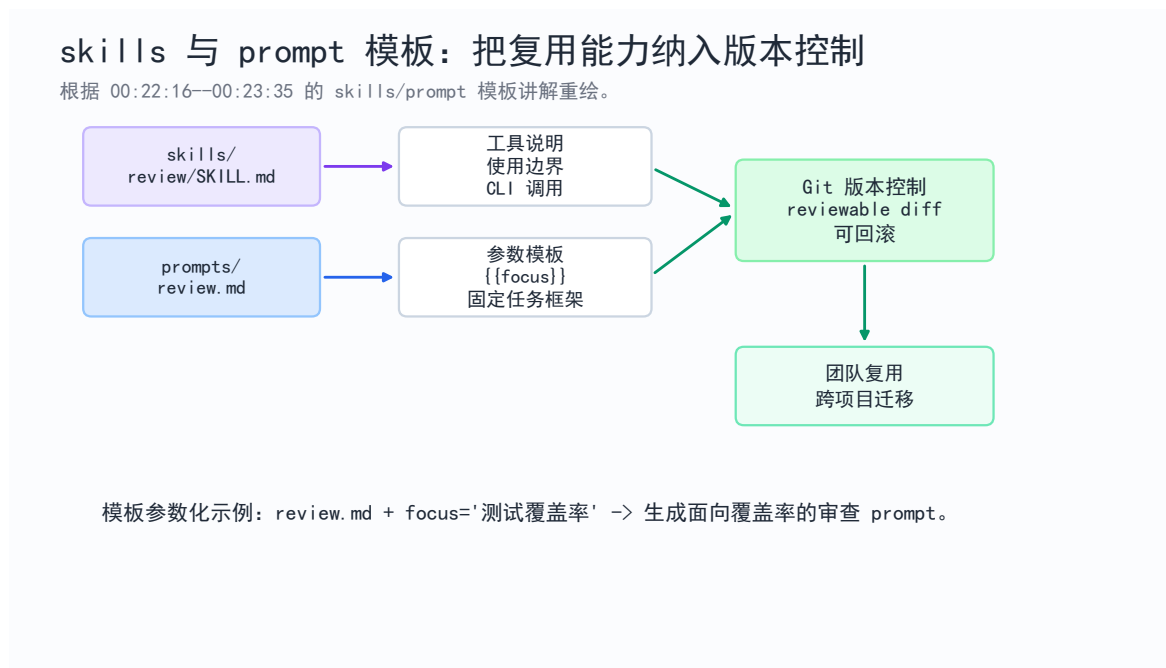


图 20: skills/prompt 模板版本化图 (根据视频讲解重绘): 能力包和提示模板应像代码一样被审查、复用和回滚。²⁰

4.7 本章小结

本章定义了 Context engineering: 它是主动设计模型上下文输入的工程活动。Pi Agent 提供了几类入口: `AGENTS.md` 承载项目级说明, `system.md` 覆盖默认 system prompt,

`append-system.md` 追加规则，`compaction` 用摘要延长会话，`skills` 把任务方法和工具封装成能力包，`prompt` 模板复用固定任务框架。核心取舍很硬：上下文窗口有限，规则、历史、工具说明和当前材料会相互挤占。会做上下文工程的人，会把稳定规则沉淀到正确文件，把短期目标留在当前 `prompt`，把完整历史交给 `session JSONL`，把继续推理交给必要摘要。

5 Extensions、Packages 与可编程安全边界

前几章已经把 Pi Agent 的极简内核、模型接入、`session` 历史和上下文工程铺开。到这一节，问题变成：当默认能力太少，用户怎样加东西；当 agent 能运行命令，用户又怎样把危险边界写进系统本身。Pi 的回答集中在 `extension API` 上。它把“想加一个能力”从口头约定变成可注册、可测试、可分发的代码。

5.1 Extension 为什么是 Pi 的扩展中枢

`Extension` 是 Pi Agent 的功能扩展机制，可注册工具、命令、快捷键、事件响应、主题和压缩策略。更直观地说，它像一块可插拔的电路板：Pi 的主体保持很小，额外能力通过扩展插槽接入。这样做的好处很硬核：核心少，默认上下文负担低；需要外部能力时，再把能力变成普通代码装进去。

`extension API` 是 Pi 暴露给扩展作者的编程接口。API，即 `application programming interface`，可以理解为一组约定好的“插孔”：扩展通过这些插孔告诉 Pi，自己能监听哪些事件、提供哪些 `tool`、增加哪些命令、改变哪些 UI 行为。这里应区分 `extension` 与前文的 `skill`。`skill` 更像“能力说明书 + 可调用脚本”的上下文资产，`extension` 则直接进入 Pi Agent 的运行时，参与事件和工具调用。

这种设计背后的动机很朴素：agent 产品一旦把所有能力内建进去，就会迅速膨胀。每个内建能力都带来维护成本、配置复杂度和潜在上下文占用。Pi 的取舍是把主体做成小内核，把差异化需求推给 `extension`。用户需要 `weather`、代码审查、数据库查询、公司内部命令，都可以沿同一条扩展路径处理。

5.2 事件、命令与工具：从 Session Started 到 Weather Tool

`extension API` 的第一类入口是事件响应。事件可以理解为 Pi 运行过程中的信号，例如 `session` 启动、`tool call` 发生、消息写入或 UI 状态改变。视频里的示例监听了 `session started`，并在 `session` 启动时改变 UI，提醒用户当前扩展已经生效。这个例子看似轻量，教学价值在于说明 `extension` 可以站在 Pi 的生命周期里观察和反应，能力范围超出孤立命令。

第二类入口是命令。命令面向人，常通过斜杠命令或快捷入口触发，用来做明确的操作。比如某个 package 装入后提供 /do 之类的命令，用户可以直接在交互式 CLI 中调用。命令的价值在于减少重复 prompt，把高频动作变成稳定入口。

第三类入口是 tool。tool 是 agent 可调用的具体能力接口，例如前文已有的 bash，也包括视频里的 greet 和 weather。tool 面向模型：用户提出任务后，模型判断是否需要调用某个 tool，并按 tool schema 填入参数。这个结构类似函数调用：

$$\text{tool result} = f(\text{validated arguments})$$

其中 f 是扩展提供的外部能力，参数由 agent 根据用户意图生成，再由运行时传给扩展。

greet 示例把一个名字变成问候语，它的意义在于展示最小 tool 的骨架：名称、描述、参数、执行函数。**weather** 示例进一步说明 tool 可以访问 URL，把城市名作为参数编码进请求，再把天气结果返回给 agent。由此可见，tool 的边界在于“把外部世界包装成模型可理解、可调用、可返回结果的接口”。

5.3 把安全策略写成拦截逻辑

扩展最关键的工程价值，出现在危险 bash 命令的拦截示例中。前文谈过 sandbox 和人工确认，这一节展示另一层防线：在 tool call 事件发生时，扩展检查即将调用的 tool 名称和参数。如果 tool 名称是 bash，并且命令文本包含删除文件或目录的危险模式，扩展就阻止执行。

这相当于给 agent 的动作链加了一个 guard：

$$\text{tool call} \rightarrow \text{guard policy} \rightarrow \{\text{allow}, \text{block}\}$$

guard policy 是可执行策略，区别于只写在文档里的提醒。文档提醒依赖人记得看，拦截逻辑会在每次 tool call 前触发。边界在于，字符串匹配只能覆盖明确写出的风险模式；更严谨的安全策略还需要命令解析、路径白名单、权限分级和审计日志。视频示例的价值是打开方法：把“哪些事不该做”写成运行时规则。

程序员读者需要补上一层背景：安全策略通常有三种形态。第一种是制度性规则，例如团队约定不要运行删除命令。第二种是交互性规则，例如执行前弹窗确认。第三种是可执行规则，例如扩展拦截 bash。Pi 的 extension API 让第三种规则进入用户自己的工具链。它不会消灭风险，但会把风险从“靠记忆”推进到“靠代码持续检查”。

5.4 从 examples 到自举式扩展开发

视频强调 examples 目录和文档面向 agent 编写。这一点很容易被低估。传统文档主要面向人类读者，好的 agent 文档还要让模型能照着完成任务：目录结构清楚、示例可

复制、API 边界明确、失败模式可定位。这样一来，用户可以让 Pi 自己阅读扩展文档，再生成或修改 extension。

这就是自举式扩展开发：Pi 作为工具，读取 Pi 的扩展文档，写出 Pi 的扩展。自举在计算机科学里常指系统用自身能力构造或改进自身，像编译器用旧版本编译新版本。这里的自举没有那么形式化，但直觉相同：一旦文档、示例和运行时接口足够规整，agent 就能参与扩展自己的工作面。

操作上，用户可以先从 `examples/extensions` 找一个最接近目标的例子，复制为本地 extension，在当前文件夹直接加载测试。直接加载本地扩展的意义是缩短反馈环：写一点、跑一次、看事件或 tool 是否生效。等能力稳定后，再考虑打包、发布和团队复用。

5.5 Packages 与 Themes：生态和外观的可定制性

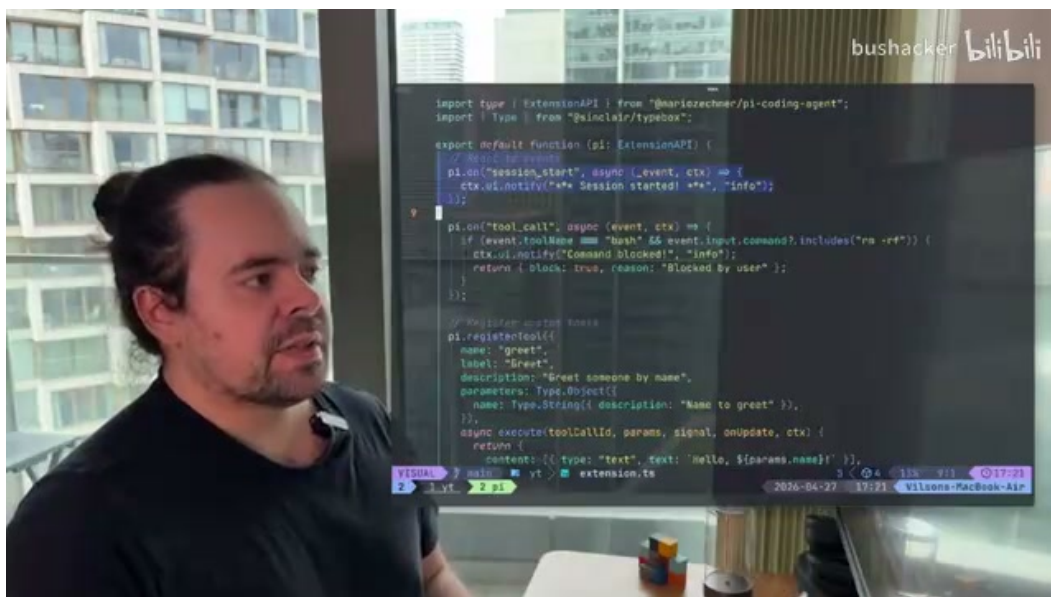
package 是可安装的扩展包，可来自 npm 或 Git 仓库。npm，即 Node Package Manager，是 JavaScript/TypeScript 生态常用的包管理系统；它像一个集中货架，用户通过包名安装发布好的扩展。Git package 指直接从 Git 仓库安装的包；Git 是分布式版本控制系统，适合安装仍在快速迭代、尚未发布到 npm 的扩展。

package 让 extension 从“本地脚本”变成“可传播资产”。一个扩展如果只存在于某台机器，它解决的是个人问题；打成 package 后，它可以在其他 Pi 会话、其他项目、其他同事机器上安装。包管理还带来列表、移除、配置、更新和本地安装位置这些日常维护动作。视频中提到包可能安装在 Pi 的 Git 子文件夹、全局 npm 位置，也可以通过本地参数放进当前项目的 Pi 子文件夹。这些位置差异会影响复现性：项目内安装更适合团队共享，全局安装更适合个人常用工具。

theme 是 Pi Agent 的外观配置，控制界面的颜色、视觉风格和局部 UI 表现。主题听起来像装饰，但在 CLI 工具里也有实际作用：长时间工作时，配色影响可读性；团队演示时，主题影响识别度；截图进入文档时，主题影响教学清晰度。视频里的 Matrix 风格主题说明 theme 也能成为 extension 生态的一部分，甚至可以由 Pi 根据文档生成。

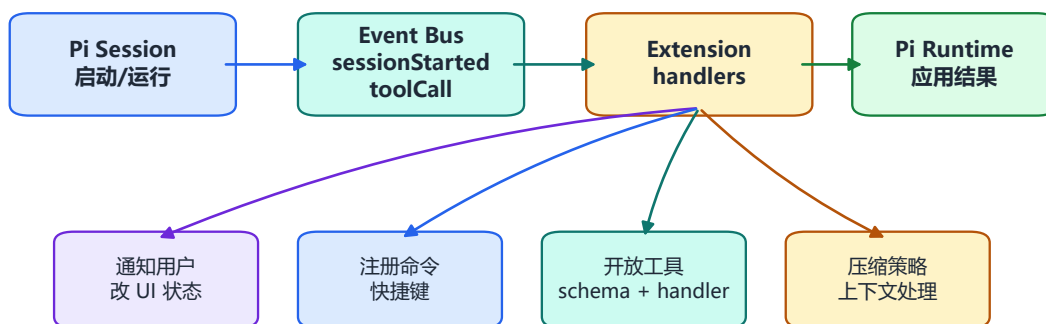
5.6 关键画面与结构图

本节图块应承担三件事：第一，用 00:23:39–00:24:33 的画面说明 extension API 与 `session started` 事件；第二，用 00:24:34–00:25:23 的画面展示危险 `bash` 命令被拦截；第三，用 00:25:25–00:26:38 的画面重绘 `greet/weather` tool schema 与调用流程。由于源视频只有 360P，代码小字建议由 figure agent 用截图作 provenance，再用流程图和表格重绘教学主体。

图 21: 来源截图: extension API 中的 session 事件处理示例²¹

Extension 事件流: 把 Pi 的运行时暴露给扩展

事件像系统里的脉搏, extension 监听它, 再注入命令、工具或 UI 行为。



核心含义: extension 覆盖一组可编程挂钩, 范围超过单个插件点。

图 22: Extension 事件流图 (根据视频讲解重绘) ²²

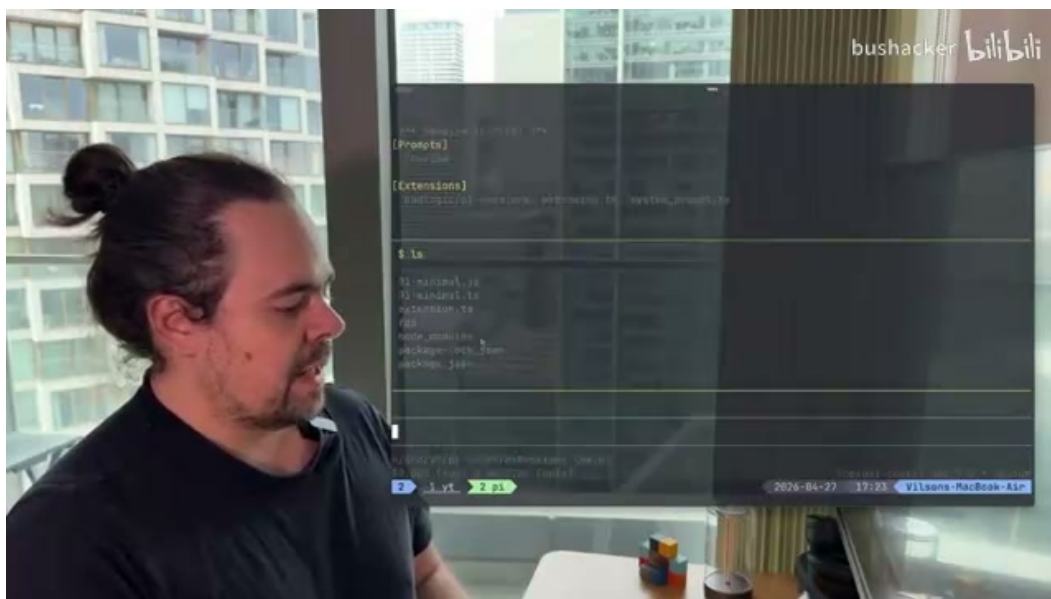
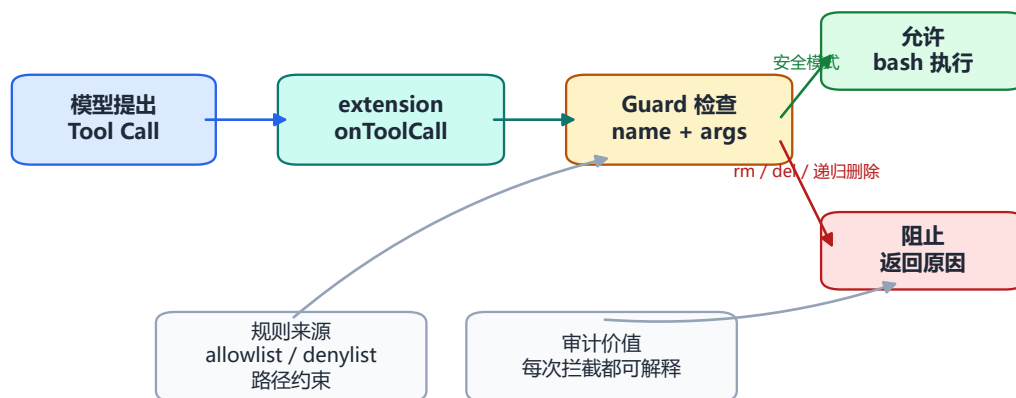


图 23: 来源截图: 扩展拦截危险 bash 命令的演示画面²³

安全流程: tool call 进入 guard 后才落地执行

危险命令从“人工盯着看”变成“代码先拦一遍”，边界更清楚。



guard 的价值在于把安全策略放进执行链路，而非停留在口头规范。

图 24: 安全流程图: 从 tool call 到 guard 再到 allow/block (根据视频讲解重绘)²⁴

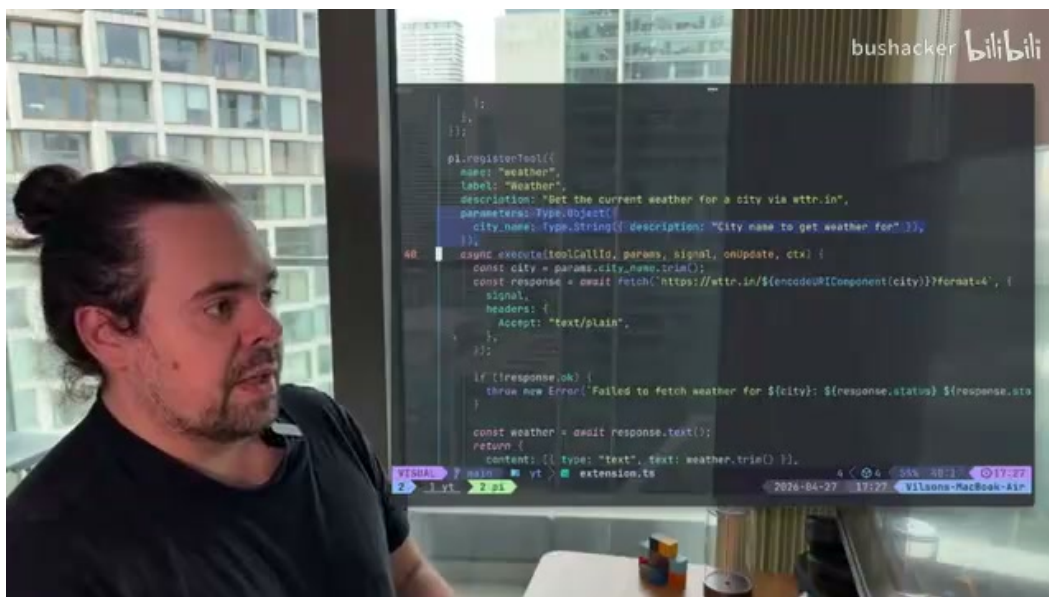
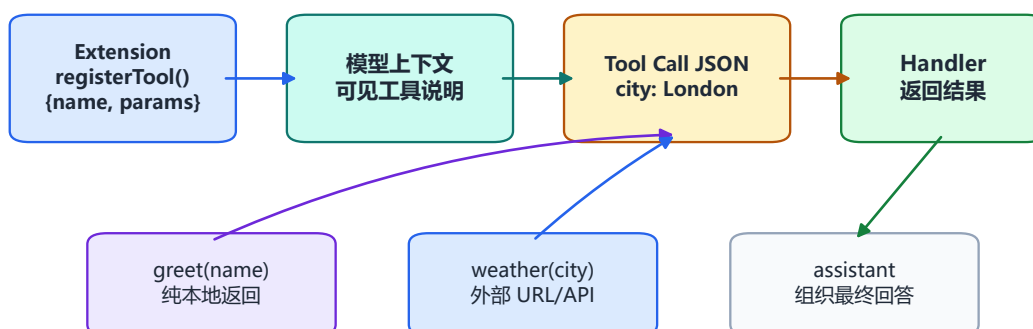


图 25: 来源截图: 通过 extension API 注册 weather 自定义 tool 的代码画面²⁵

自定义 Tool: schema 决定模型怎样调用外部能力

greet/weather 示例的关键在于: 先声明接口, 再让 handler 承接真实执行。



schema 像插座规格: 模型按规格发起调用, 扩展负责把电接到真实系统。

图 26: 自定义 tool schema 与调用链路 (根据视频讲解重绘) ²⁶

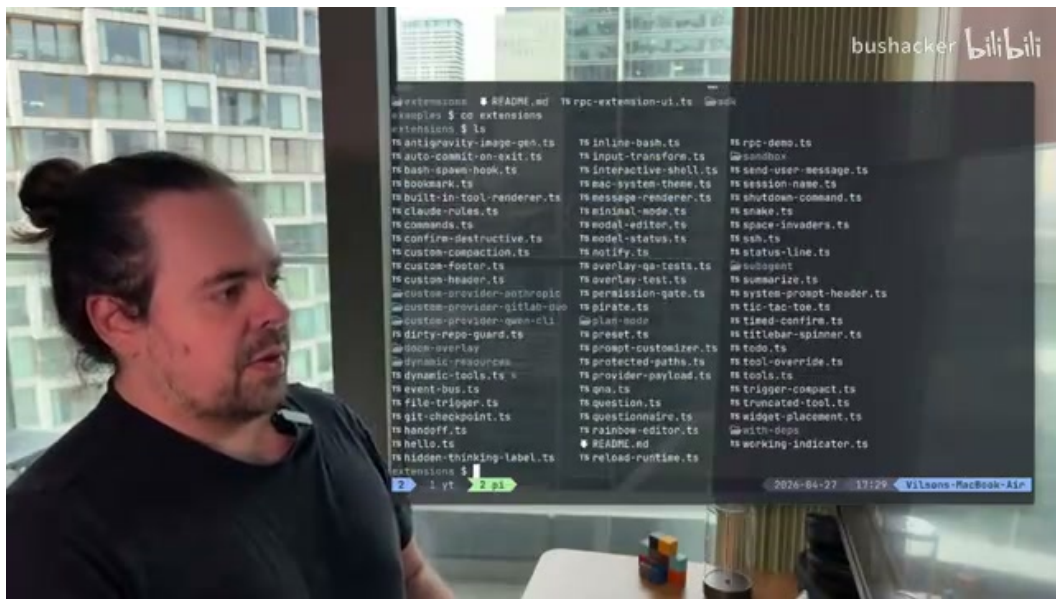


图 27: 来源截图: examples/extensions 目录树, 展示扩展开发资料的位置²⁷

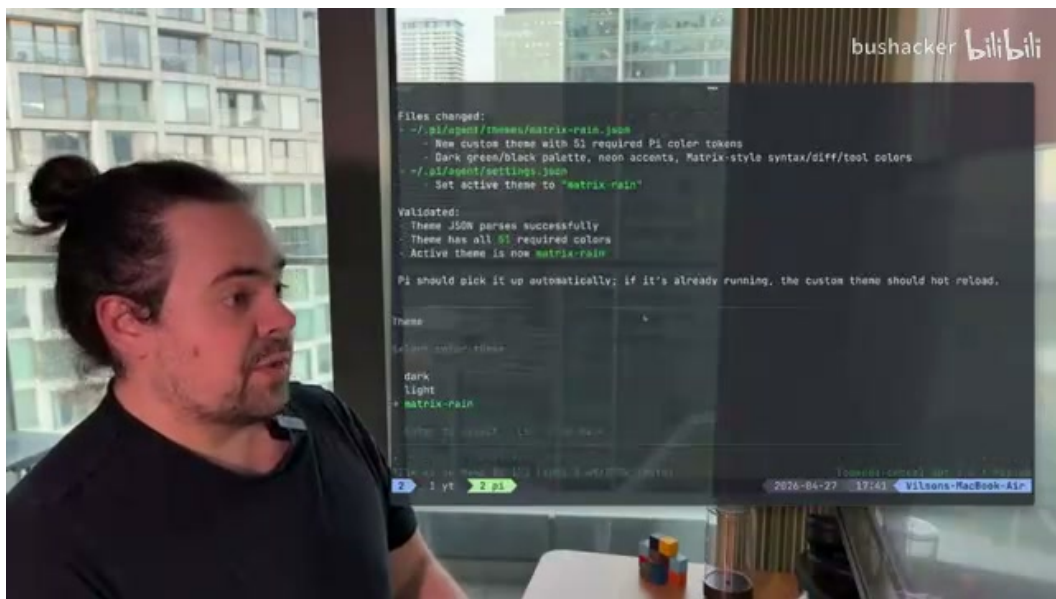


图 28: 来源截图: package 与 theme 配置后的 Matrix 主题效果²⁸

Packages 与 Themes: 扩展生态的安装面和外观面

package 负责能力分发, theme 负责交互外观; examples/docs 让 agent 能继续自举。

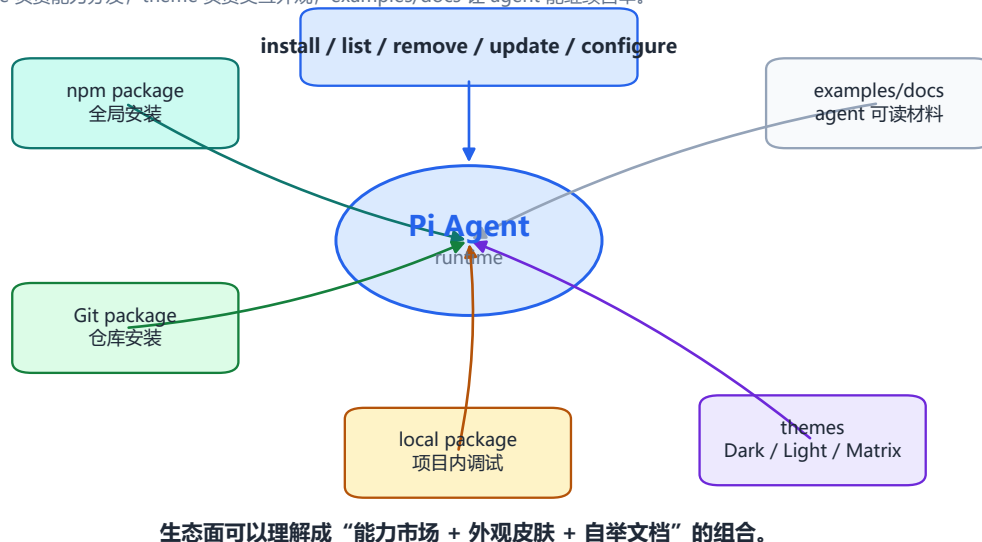


图 29: Packages 与 Themes 生态图 (根据视频讲解重绘) ²⁹

5.7 本章小结

本章把 Pi Agent 的扩展生态压缩成一条主线: extension 是运行时插槽, extension API 是插槽协议, tool 是模型可调用的能力接口, package 是可分发的扩展单元, theme 是外观和 UI 行为的可定制资源。npm 与 Git package 则对应两种常见分发路径。

更重要的是, 扩展机制把 Pi 的极简主义从“少功能”推进到“可选择地加功能”。用户可以加 weather 这样的外部能力, 也可以把危险命令拦截写成 guard policy。Pi 的小内核因此不贫乏, 关键在于它把能力、安全和外观都交给可编程的组合机制。

6 多种集成形态与社区分支：把 Pi 改造成自己的工具

前一节说明 Pi Agent 如何通过 extension 和 package 增加内部能力。本节转向另一个方向: Pi 怎样被外部程序驱动。交互式 CLI 适合人坐在终端前和 agent 来回协作, 但程序员常常还需要批处理、日志分析、脚本集成、服务端编排和语言移植。视频结尾的重点在这里收束: Pi 的价值覆盖命令行界面, 也覆盖拆开、接入、包装和 hack 的工程空间。

6.1 从交互式 CLI 到批处理

print mode 是 Pi Agent 的非交互式调用方式，用于批处理、管道和一次性 prompt。交互式 CLI 像结对编程，用户不断补充问题、确认方向、观察 agent 的动作；print mode 像 Unix 命令，输入一次，输出一次，适合放进脚本和 shell pipeline。

从操作上看，print mode 通常通过类似 `pi -p "prompt"` 的参数触发。用户可以让 Pi 分析当前文件夹里有哪些文件，也可以把错误日志、测试输出或构建报告通过管道传给 Pi，让它解释失败原因。管道的价值在于组合：前一个命令产生文本，Pi 接收文本并生成解释，后一个命令还可以继续处理解释结果。这个模式延续了 Unix 哲学中的“小工具串起来做大事”。

这里应区分两类使用场景。人工探索适合交互式 CLI，因为用户会不断修正问题；确定性较强的重复任务适合 print mode，例如“总结今天的错误日志”“解释某个 CI 输出”“把一段 commit diff 转成审查要点”。前者需要对话弹性，后者需要可脚本化入口。

6.2 JSON Event Stream 与 RPC

JSON event stream 是将 session 事件作为 JSON 流输出的模式，便于程序消费。更具体地说，它把用户消息、assistant 消息、tool call、tool result 等运行过程拆成一行行 JSON 对象输出到标准输出。每行一个对象的格式通常称为 JSON Lines，与前文 session 历史中的 JSONL 思路一致：它适合追加写入、逐行解析和流式处理。

JSON event stream 的意义在于可观测性。人类看最终回答，程序更关心过程事件：模型何时开始回答、调用了哪个 tool、tool 返回什么、是否出现错误、token 和成本如何变化。把这些事件变成 JSON 流后，外部程序就可以做日志归档、实时面板、审计记录、告警触发或二次分析。

RPC 是 Remote Procedure Call，此处指外部程序通过标准输入输出驱动 Pi Agent。它的直觉是：Python、Go、Rust 或其他语言的程序把 Pi 当作一个子进程启动，通过 stdin 发送请求，通过 stdout 读取响应。调用者像调用远端函数一样调用 Pi，虽然底层通信发生在本机进程之间。

RPC 与 JSON event stream 经常搭配出现。RPC 解决“怎样把请求发进去”，JSON event stream 解决“怎样把过程和结果读出来”。如果把 Pi 看成一台小型 agent 引擎，RPC 是方向盘，事件流是仪表盘。方向盘负责控制，仪表盘负责观察。

6.3 SDK：从零构造自己的 Agent Session

SDK 是 Software Development Kit，用函数和类直接构造 Pi Agent 能力的开发接口。RPC 把 Pi 作为外部子进程驱动，SDK 则让开发者在同一语言生态内直接创建 agent

session、写入 prompt、读取消息和处理结果。它更接近库调用，适合开发者把 Pi 的能力嵌进自己的应用。

视频中的 SDK 示例创建了一个 agent session，写入 prompt，让它列出文件，并得到包含 user message 与 assistant message 的 JSON 输出。这个流程可以抽象成四步：

1. 创建 session，把模型、工具和配置装配起来。
2. 写入用户 prompt，作为本轮任务输入。
3. 运行 agent 循环，让模型生成回答或调用 tool。
4. 消费结构化输出，用于 UI 展示、日志记录或后续程序逻辑。

SDK 的优势是控制粒度更细。开发者可以决定何时创建 session、如何接入自定义 tool、怎样保存消息、如何处理错误和重试。它的成本也更明确：需要理解 Pi 的内部对象模型。对于一次性脚本，print mode 更直接；对于跨语言集成，RPC 更通用；对于要把 agent 能力产品化的应用，SDK 更有余地。

6.4 社区分支与语言移植

lazy-pi 是社区上手包，预装多个 Pi package，用来快速获得 sub-agent、MCP 和其他扩展支持。它适合希望先体验“增强版 Pi”的用户：不用从零挑选扩展，先拿到一组常用组合，再按需求删改。它也说明 Pi 生态可以出现发行版式的包装，就像 Linux 发行版把内核、包管理和默认工具组合成可用系统。

OMIP 是 Pi 的社区 fork，视频提到它把部分工具用 Rust 实现以提高效率。fork 在开源语境里指从一个项目分叉出独立演化版本。OMIP 的意义包含性能优化，也说明 Pi 的结构足够清楚，别人可以拿走思想和代码，沿自己的工程判断重组。

py2rust 是社区把 Pi/PY 思路迁移到 Rust 的方向。Rust 常被用于强调内存安全、性能和可维护性，语言移植会迫使项目重新表达核心抽象：session 怎样表示，tool 怎样注册，事件怎样流动，配置怎样加载。移植的边界在于概念重建，它会检验一个系统的概念是否足够干净。

MODEX 是讲者启动的 AI coding 工具项目，借鉴 Pi 的一些思路，并尝试在另一种编程语言和工程栈里重新组织 AI coding 工具。lazy-pi、OMIP、py2rust、MODEX 放在一起看，说明 Pi Agent 的影响力来自一组可迁移的设计取向：极简内核、可编程扩展、结构化事件和可被用户 hack 的边界。

6.5 最终综合：极简主义怎样变成可塑性

视频最后的总结可以压成五个问题。

第一，Pi 是什么？Pi Agent 是一个极简编码 agent：它以短 system prompt、少量核心工具、session 历史和 extension 体系为主要结构。它把许多复杂能力留给模型已有的编码行为和用户自己的组合。

第二，Pi 能做什么？它能读写文件、运行命令、组织会话、接入不同模型、管理上下文、调用 tool、加载扩展、安装 package，并通过 print mode、JSON event stream、RPC 和 SDK 进入自动化流程。它的能力范围更像一组可拼接的接口。

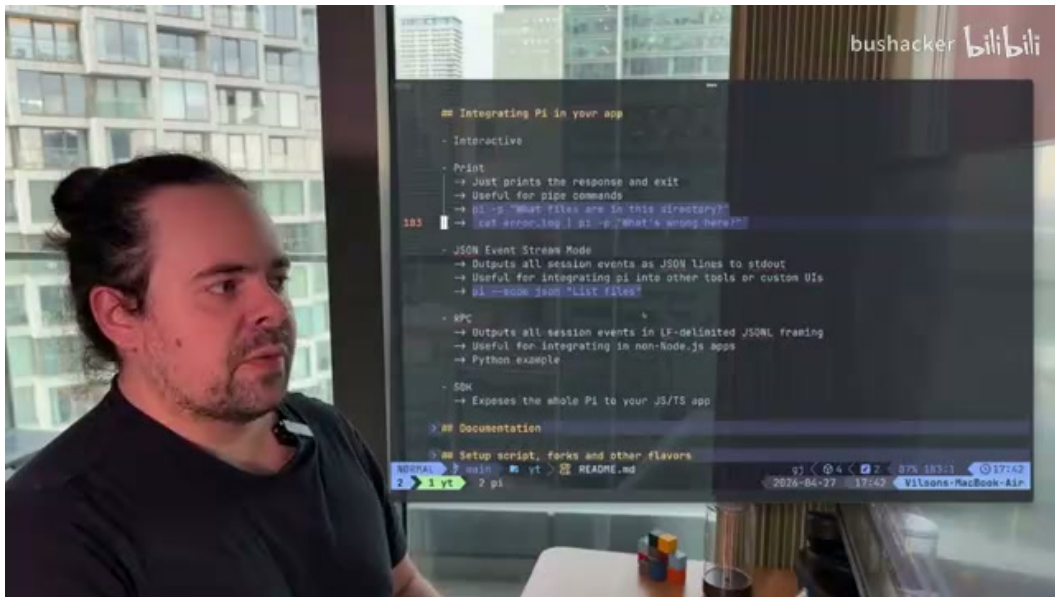
第三，怎样探索？用户可以从交互式 CLI 入手，查看命令帮助、切换 provider、观察 session 树、阅读 examples 和文档。探索 Pi 的正确姿势是把它当作可检查的工程系统：看配置、看日志、看事件、看扩展示例。

第四，怎样扩展？从本地 extension 开始，先监听简单事件或写一个最小 tool，再逐步加入参数校验、错误处理和安全策略。能力稳定后，把它打成 npm package 或 Git package，供其他 session 和团队成员复用。

第五，怎样按自己的需求 hack？hackable 在这里指可塑性和可组合性。用户可以写规则文件改变上下文，可以用 extension 增加能力，可以用 package 分享能力，可以用 print mode 和 RPC 接入脚本，也可以用 SDK 把 Pi 嵌进产品。Pi 的极简主义最终指向更少预设，把决定权交回给会写代码的人。

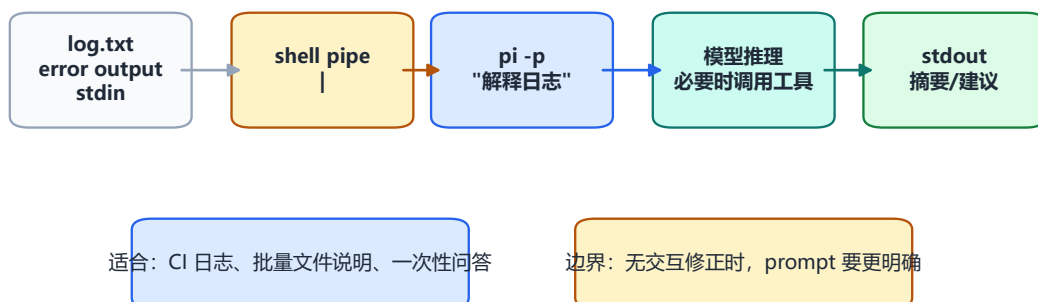
6.6 关键画面与结构图

本节图块建议覆盖四类结构：00:29:53–00:30:30 的 print mode 与 shell 管道，00:30:31–00:31:02 的 JSON event stream，00:31:03–00:31:41 的 RPC 子进程通信，00:31:42–00:32:43 的 SDK 创建 session。00:32:44–00:34:13 可重绘社区生态地图，展示 lazy-pi、OMIP、py2rust、MODEX 如何围绕 Pi 的思想分叉。结尾可用综合图回收“极简内核 -> 上下文工程 -> 扩展 -> 集成”的主线。

图 30: 来源截图: print mode 的命令行调用画面³⁰

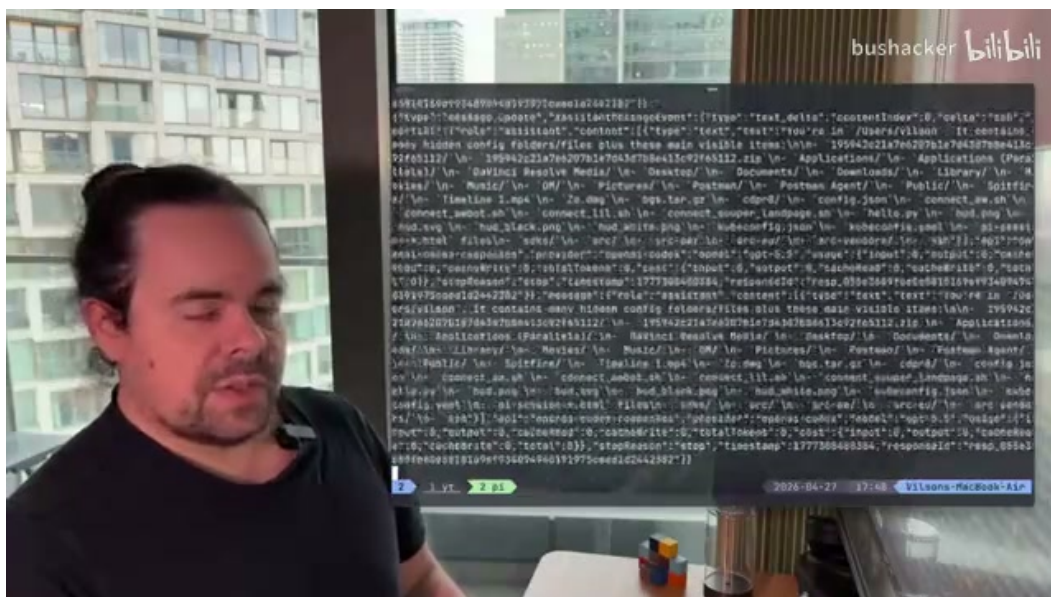
Print mode 管道：让 Pi 成为 shell 工作流的一段

一次性 prompt 与标准输入结合后，Pi 可以参与批处理、日志解释和脚本流水线。



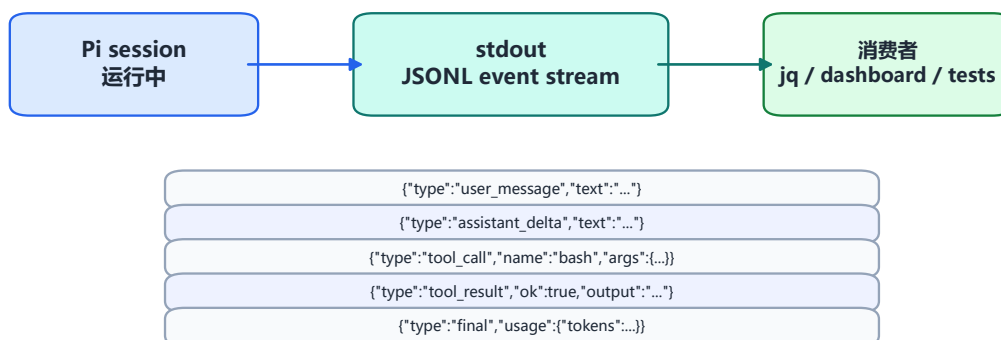
print mode 的本质是把 agent 能力压缩成可组合的 CLI 过滤器。

图 31: Print mode 管道图（根据视频讲解重绘）³¹

图 32: 来源截图: JSON event stream 输出演示画面³²

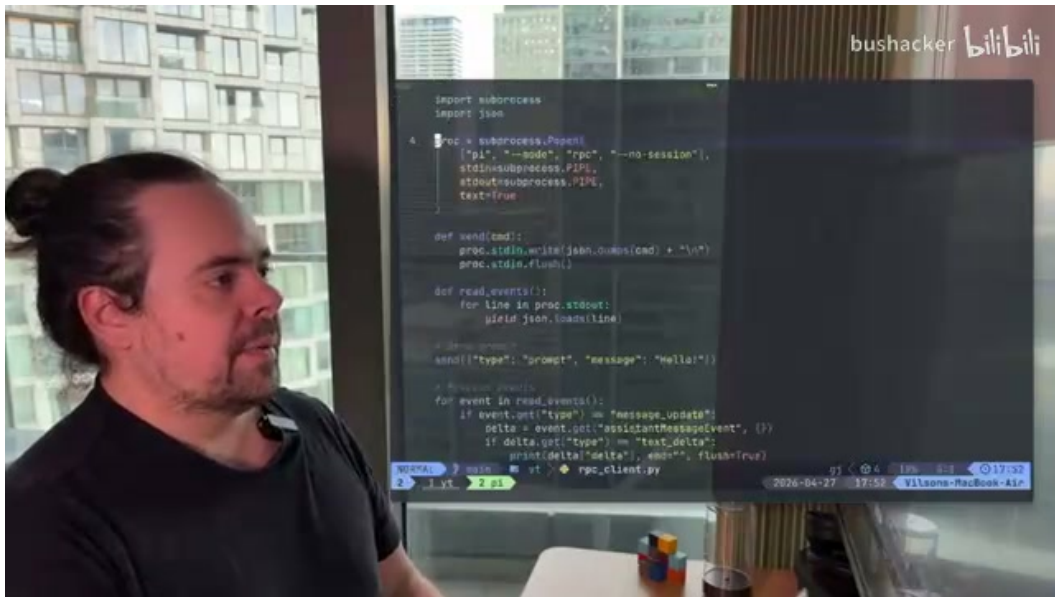
JSON Event Stream: 把 session 变成可消费的 JSON Lines

每一行都是事件对象，程序可以边读边处理。



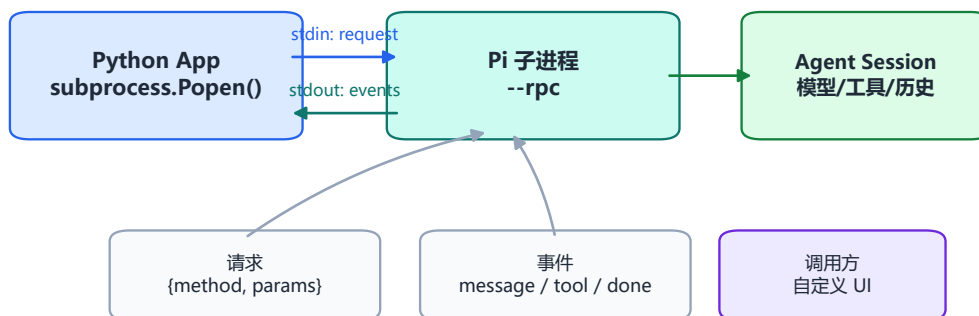
JSONL 的优势是“追加写入 + 流式读取”，适合日志系统和外部编排。

图 33: JSON event stream 图: session 事件的 JSON Lines 输出（根据视频讲解重绘）³³

图 34: 来源截图: RPC 子进程通信示例代码画面³⁴

RPC stdin/stdout: 非 Node.js 程序通过子进程驱动 Pi

RPC 模式把 Pi 包成一个可对话的后台进程。



RPC 关注进程边界：调用方保留自己的语言栈，Pi 负责 agent 执行。

图 35: RPC stdin/stdout 通信图（根据视频讲解重绘）³⁵

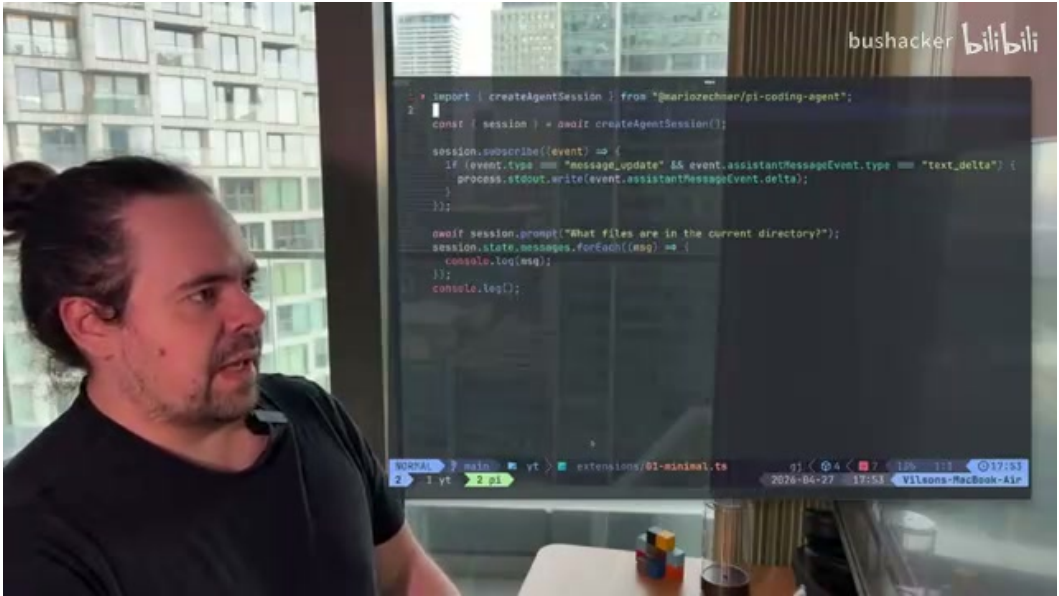


图 36：来源截图：通过 SDK 创建 agent session 的代码画面³⁶

SDK 调用序列：从函数和类层面构造 Agent Session

SDK 适合把 Pi 嵌进更大的应用，而非只当命令行程序调用。

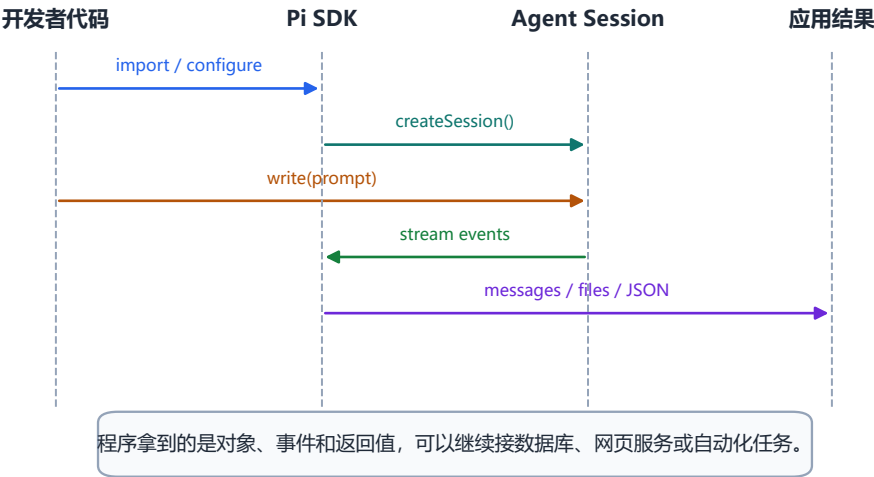


图 37：SDK 调用序列图（根据视频讲解重绘）³⁷



图 38：来源截图：OMIP 等社区分支与移植项目的演示画面³⁸

社区生态地图：包装、移植、重写与再发明

生态含义：PI 的接口足够小，社区才容易包一层、换语言、换目标场景。
开源项目的生命力来自可分叉性，同一思想可以被放进不同语言和平台。

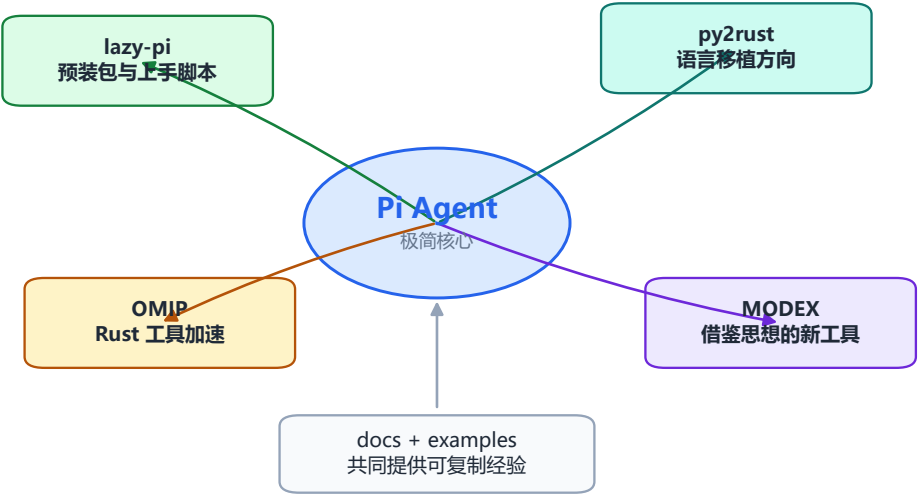
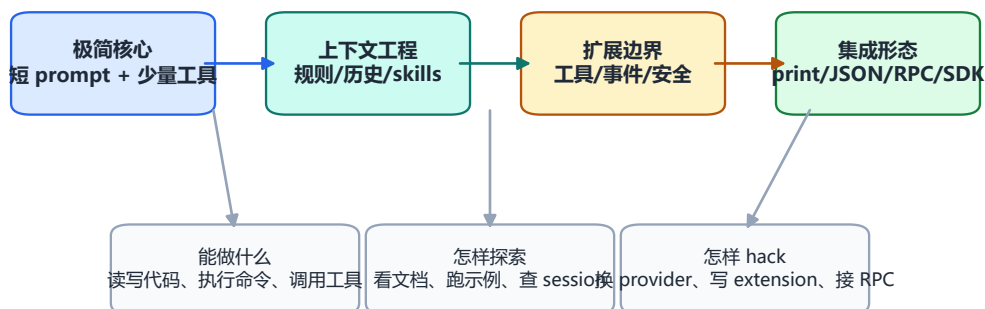


图 39：社区生态地图（根据视频讲解重绘）³⁹

最终综合：极简主义怎样变成可塑性

视频结尾回收的主线可以压缩成四级能力梯。



小核心的收益来自清晰边界：每个方向都能被普通工程手段接管。

图 40：最终综合图：极简核心到可塑工具链（根据视频讲解重绘）⁴⁰

6.7 本章小结

本章把 Pi Agent 从“一个终端里的 agent”推进为“可被程序驱动的 agent 引擎”。print mode 让它进入批处理和管道，JSON event stream 让过程可被程序消费，RPC 让非 Node.js 应用通过子进程驱动 Pi，SDK 让开发者从函数和类层面构造自己的 agent session。

社区分支则证明了同一个核心思想可以被包装、移植、重写和再发明。读完整个视频，读者应能带走一个直接判断：Pi 的力量来自小内核与开放接口之间的张力。理解这点之后，用户就能决定何时保持简单，何时加扩展，何时接入自动化，何时按自己的需求 hack 出一套真正顺手的工具。

7 总结与延伸：极简主义怎样落到工程手上

这支视频的结尾回到一个很朴素的问题：Pi Agent 到底给程序员留下了什么。讲者把重点放在一条可操作路径上：先理解 Pi 是什么，再理解它能做什么，接着学会探索功能、扩展接口，并把它改造成适合自己工作流的工具。这个收束很关键，因为 Pi Agent 的价值不仅来自“功能少”，也来自每个保留下来的部件都可以继续组合。

全篇核心压缩

Pi Agent 的主张可以压缩成一句工程判断：当模型已经通过训练学会基本编码循环时，agent 框架的默认形态应尽量轻，复杂能力通过文件、CLI、extension、SDK 和外部工具按需接入。这里的“轻”指上下文占用轻、认知负担轻、改造成本低。

7.1 从一个极简内核长出完整 workflow

前六章形成了一条递进链路。第一章给出 Pi Agent 的内核：短 system prompt、少量工具、bash 与文件系统。第二章说明它如何连接 provider、订阅模型、OpenRouter 和本地模型。第三章把每次对话变成 session 树，使探索路径可恢复、可分叉、可审计。第四章进入 context engineering，把 AGENTS.md、system.md、append-system.md、skills 和 prompt 模板纳入上下文管理。第五章讨论 extension 和 package，让安全策略、工具调用和主题定制进入可编程层。第六章把 Pi 推向 print mode、JSON event stream、RPC、SDK 和社区 fork，使它从交互式 CLI 变成可嵌入系统的一块积木。

这条链路像一把从小到大的折叠刀：闭合时很小，打开后每个工具都有明确位置。极简主义的风险在于工具不足，Pi 的解法是把扩展点保留下来。过度框架化的风险在于上下文被框架细节吞掉，Pi 的解法是让用户自己决定哪些能力进入当前任务。

7.2 上下文预算是这套设计的隐含账本

视频反复触到一个没有被写成公式的账本：模型一次工作能看到的 token 有上限。MCP server 描述、历史消息、规则文件、工具 schema、计划文本都会挤占这个空间。可以用一个轻量预算式理解：

$$C_{usable} = C - C_{mcp} - C_{history} - C_{rules}$$

- C ：模型的总上下文窗口。
- C_{mcp} ：MCP 相关描述和接口说明占用的上下文。
- $C_{history}$ ：对话历史、session 摘要和上下文延续材料占用的上下文。
- C_{rules} ：AGENTS.md、system prompt、追加规则、工具说明等占用的上下文。
- C_{usable} ：真正留给当前代码、错误日志、任务推理和修改方案的空间。

上下文有真实成本

把所有能力预先塞给 agent，看起来像增强，实际会提高每轮推理的固定成本。对复杂项目来说，真正稀缺的常常是“当前任务能占多少工作台面积”。Pi Agent 的极简取向提醒读者：先保留可用上下文，再按任务引入能力。

7.3 安全、并行和计划都可以外接

讲者对 sandbox、tmux 和 Markdown 计划文件的强调，体现的是 Unix 式组合思维。安全可以交给 sandbox，后台任务和多会话观察可以交给 tmux，计划可以写成普通 Markdown 文件。这样做的好处是边界清晰：agent 框架只承担 agent 的核心循环，成熟工具继续承担它们擅长的职责。

这也解释了 Pi Agent 的 hackable。用户保有一组可被脚本、配置、extension、SDK 重新组合的零件，无需接受固定完整工作台。用户需要更多安全策略，可以写 bash 拦截；需要专用工具，可以注册 extension tool；需要程序化调用，可以用 RPC 或 SDK；需要上手包，可以看 lazy-pi 一类社区项目。

7.4 实践上的取舍

Pi Agent 的现实边界也很直接。第一，极简框架把更多选择权交给用户，用户需要懂一些 CLI、文件结构和上下文管理。第二，短 prompt 依赖模型本身已经学会编码工作流；当模型能力较弱时，外部脚手架仍然有价值。第三，extension 和 SDK 带来可塑性，也带来维护责任：接口、权限、日志和错误处理都需要被认真设计。

读者可以继续追问的三个问题

- 哪些团队任务适合极简 agent，哪些任务需要强约束流程？
- MCP、skills、extension 三者的职责边界怎样划分，才能减少重复上下文？
- 当 agent 可以通过 SDK 嵌入产品时，权限审计、成本观测和回滚机制应该放在哪一层？

7.5 本章小结

如果只带走一个结论，那就是：Pi Agent 把 agent 工程从“堆能力”拉回到“算预算”。工具越多，默认上下文越重；入口越复杂，调试和改造越难。真正强的系统常常有一个瘦内核，再通过清楚的边界向外生长。Pi Agent 给程序员的启发恰好在这里：先保留一个能读、能写、能编辑、能运行命令的核心，再把安全、并行、计划、扩展、集成这

些能力按任务挂上去。

这也是视频结尾那句“按自己的需求 hack”的实质含义。它强调 agent 是一个可塑的工程对象：能观察、能复现、能扩展、能替换、能被放进更大的自动化系统；随意拼凑不属于这里的 hack。少即是多在这里不只是口号，它是一套关于上下文、接口和责任边界的工程纪律。